

第8章 内部排序

教学 重 点

8.1 概述

8.2 插入排序

8.2.1 直接插入排序(稳定)

8.2.2 折半插入排序(稳定)

8.2.3 希尔排序(不稳定)

8.3 交换排序

8.3.1 冒泡排序(稳定)

8.3.2 快速排序(不稳定)

8.4 选择排序(不稳定)

8.4.1 简单选择排序(不稳定)

8.4.3 堆排序(不稳定)

8.5 归并排序：2-路归并排序(稳定)

8.6 基数排序(稳定)

8.7 各种内部排序方法的比较讨论



8.1 概述(P289)

1. 什么是排序?

将一组杂乱无章的数据按一定规律顺次排列起来。

存放在数据表中

按关键字非递减

数组 $R[n]$ 中包含 n 个记录: $R_1, R_2, \dots, R_n$, 对应关键字分别为 $k_1, k_2, \dots, k_n$

猴 马 鸡 猪

3 1 9 3

排序

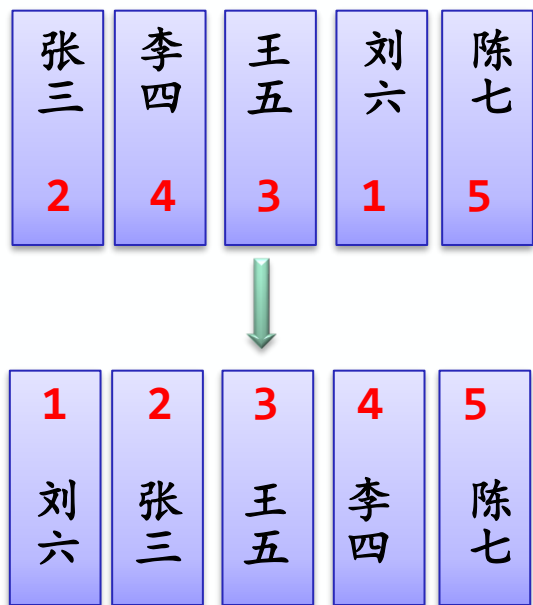
使得相应关键字序列满足非递减 $k_{p_1} \leq k_{p_2} \leq \dots \leq k_{p_n}$, 原本序列
也能成为一个按关键字有序的序列 $R_{p_1}, R_{p_2}, \dots, R_{p_n}$
马 猴 猪 鸡

2. 排序的目的是什么?

——便于查找!

排序算法的稳定性(P290)

当待排序记录中的关键字均不相同，排序的结果是**唯一**的。



排序算法的稳定性(P290)

若待排序的表中存在有多个关键字相同的记录，经过排序后这些具有相同关键字的记录之间的相对次序保持不变，则称这种排序方法是**稳定**的。

排序前

3 4 3 1 5



排序后

1 2 3 3 5

反之，若具有相同关键字的记录之间的相对次序发生变化，则称这种排序方法是**不稳定**的。

排序前

3 4 3 1 5



排序后

1 2 3 3 5

排序算法效率的评价指标(P291)

排序算法的好坏如何衡量?

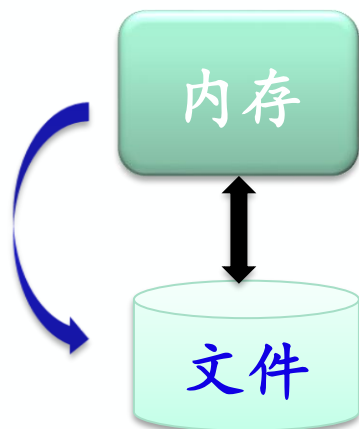
- 时间效率——排序速度（即排序所花费的全部比较次数）
- 空间效率——占内存辅助空间的大小
- 稳定性——若两个记录A和B的关键字值相等，且排序后A、B的先后次序保持不变，则称这种排序算法是稳定的。

内排序和外排序(P290)

按数据存储介质分类：

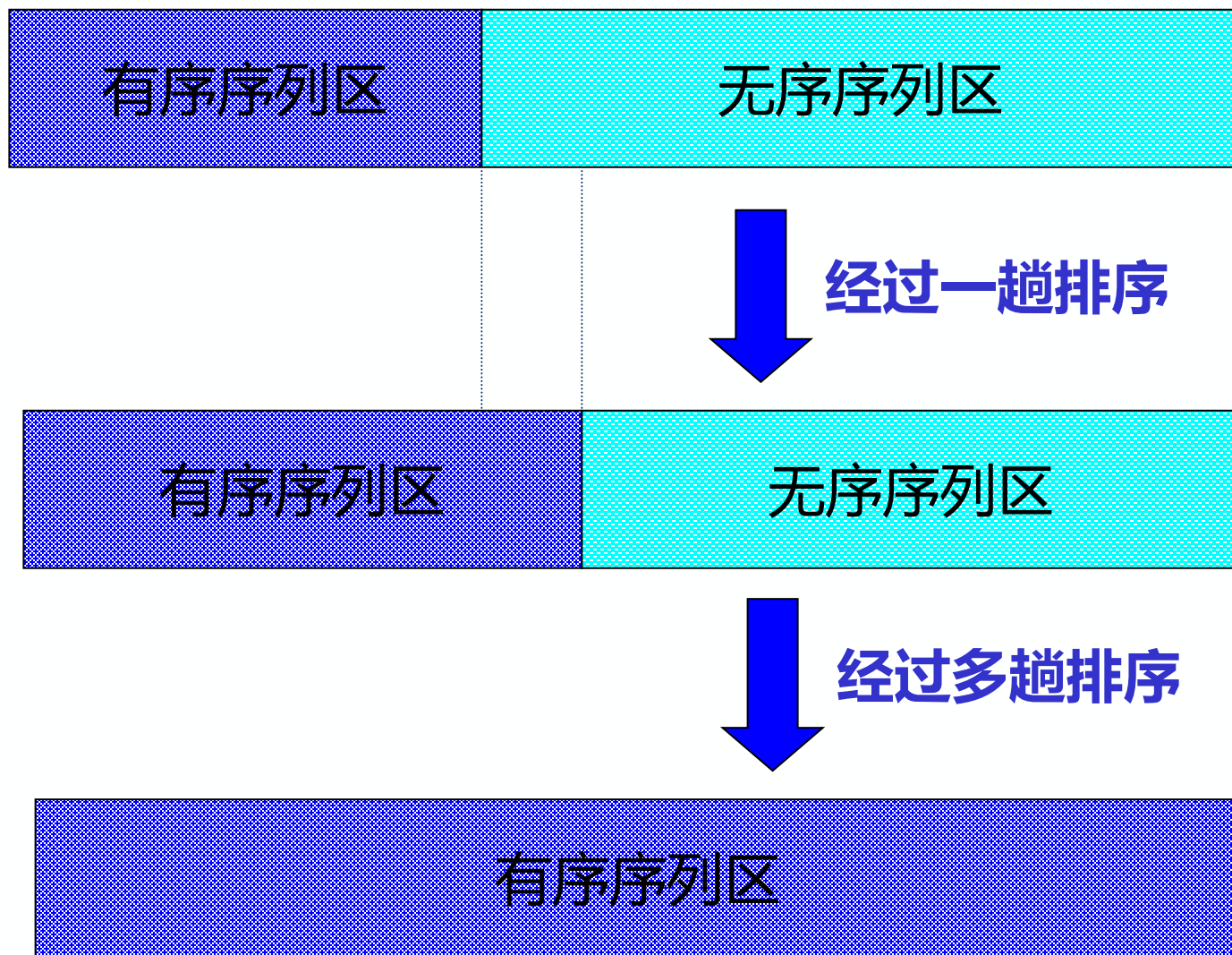
若待排序元素数量少，在排序过程中，若整个表都是放在内存中处理，**排序时不涉及数据的内、外存交换**，则称之为**内部排序**；

反之，若待排序元素数量多，以致内存一次不能容纳全部元素，排序过程中要进行数据的内、外存交换，则称之为**外部排序**。



8.1.2 内部排序的分类(P290)

内部排序的过程是一个**逐步扩大记录的有序序列长度的过程**。



8.1.2 内部排序的分类(P290)

基于逐步“扩大”有序区的不同原则，内部排序方法大致可分以下几种类型：

- 1) **插入类**：将无序子序列中的一个或几个记录“插入”到有序序列中，从而增加记录的有序子序列的长度。
- 2) **交换类**：通过“交换”无序序列中的记录从而得到其中关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。（冒泡、快速）
- 3) **选择类**：从记录的无序子序列中“选择”关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度。（简单选择排序、堆排序）
- 4) **归并类**：通过“归并”两个或两个以上的记录有序子序列，逐步增加记录有序序列的长度。（2-路归并排序）
- 5) **分配类**：唯一一类不需要进行关键字比较的方法（基数排序）

存储结构——以顺序表存储(P291)

待排序的顺序表的数据元素类型声明如下：

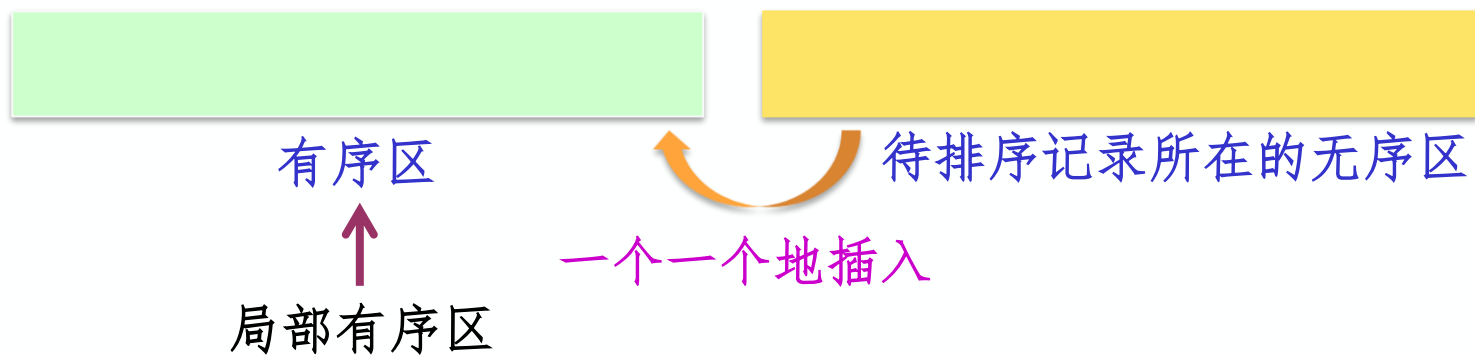
```
#define MAXSIZE 20           //假设小顺序表的最大长度为20
typedef int KeyType;         //定义关键字类型为整型类型
typedef struct               //定义记录（数据元素）的结构
{   KeyType key;             //关键字项
    InfoType data;          //其他数据项，类型为InfoType
}   RedType;                //Record Type 排序的记录类型定义
```

```
typedef struct               //定义顺序表的结构
{
    RedType R[MAXSIZE+1];    //存储顺序表，R[0]用作哨兵单元或缓冲区
    int length;              //顺序表长度
}   SqList;                 //顺序表类型
```



8.2 插入排序(P292)

基本思想：每一趟将一个待排序的关键字，按其大小，**插入到已经排好序的一组关键字的适当位置上**，直到所有待排序关键字全部插入为止。

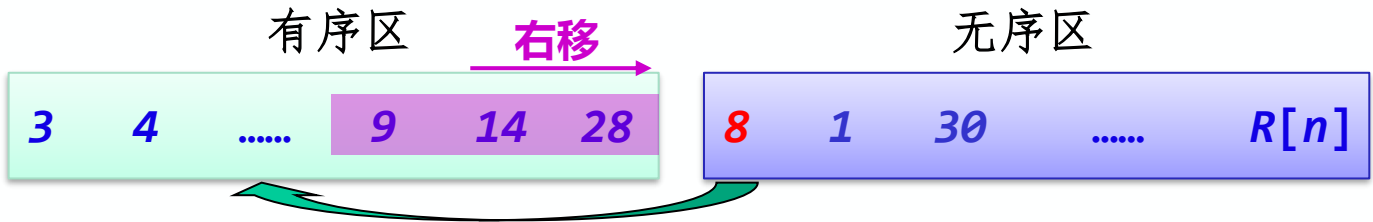


有序区类型

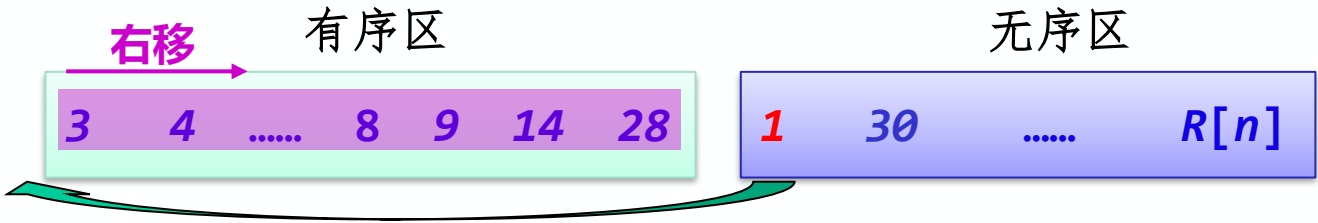
- **局部有序区**：局部有序区中元素在后面排序中会发生位置的改变。
如 $[3, 6, 8]$, $1, 7, 5$, 其中 $[3, 6, 8]$ 为局部有序区
- **全局有序区**：全局有序区中元素在后面排序中不再发生位置的改变。
如 $[2, 4, 5]$, $6, 7, 9$, 其中 $[2, 4, 5]$ 为全局有序区

插入位置的不同

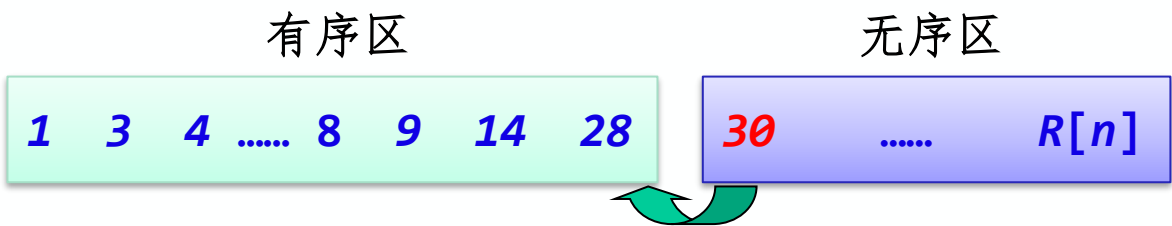
1) 插入在中间位置



2) 插入在最前面



3) 插入在最后面



如何找到插入位置呢?



插入排序

直接插入排序：顺序法定位插入位置
(基于顺序查找)

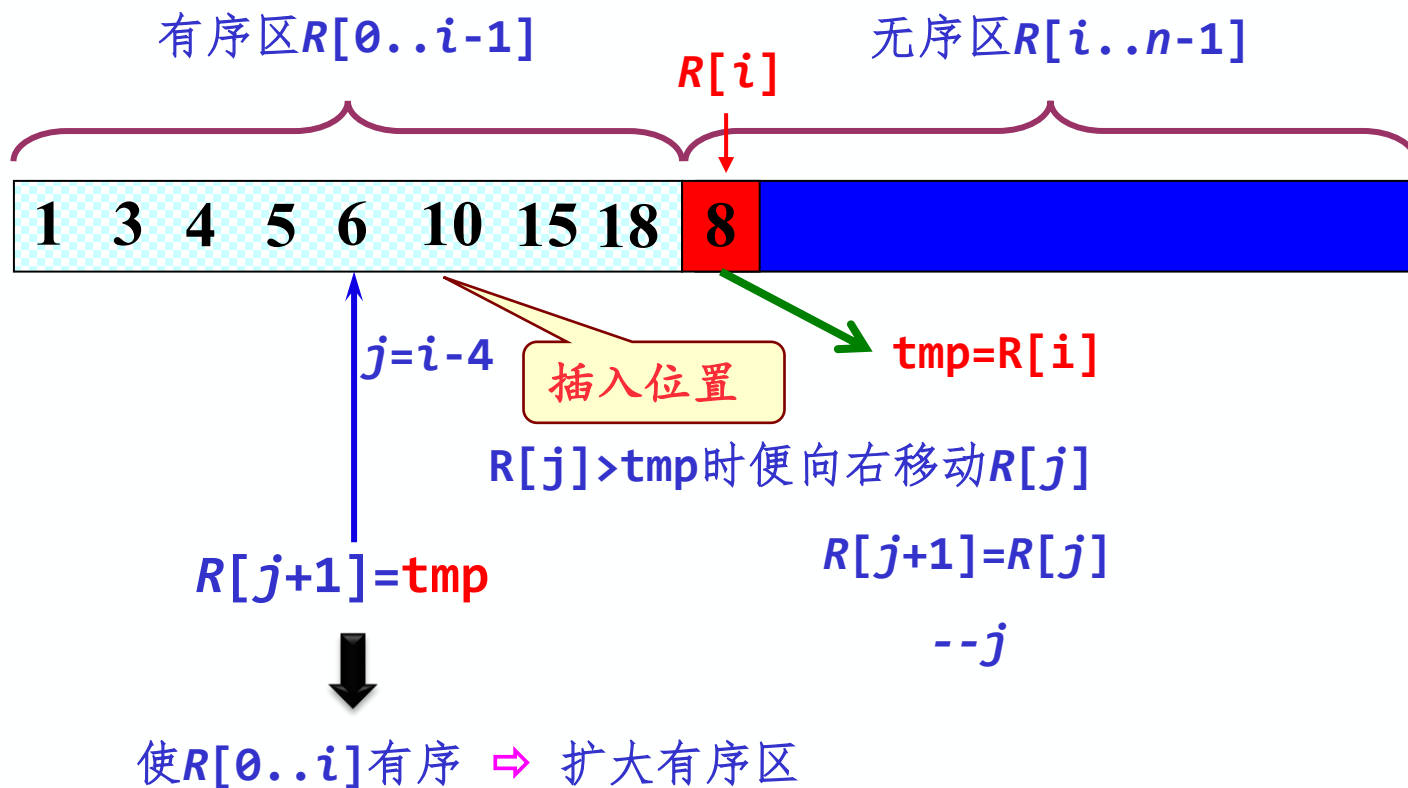
折半插入排序：二分法定位插入位置
(基于折半查找) 小改进

希尔排序：逐趟缩小增量，多遍插入排序
大改进

8.2.1 直接插入排序(P292)——临时变量法

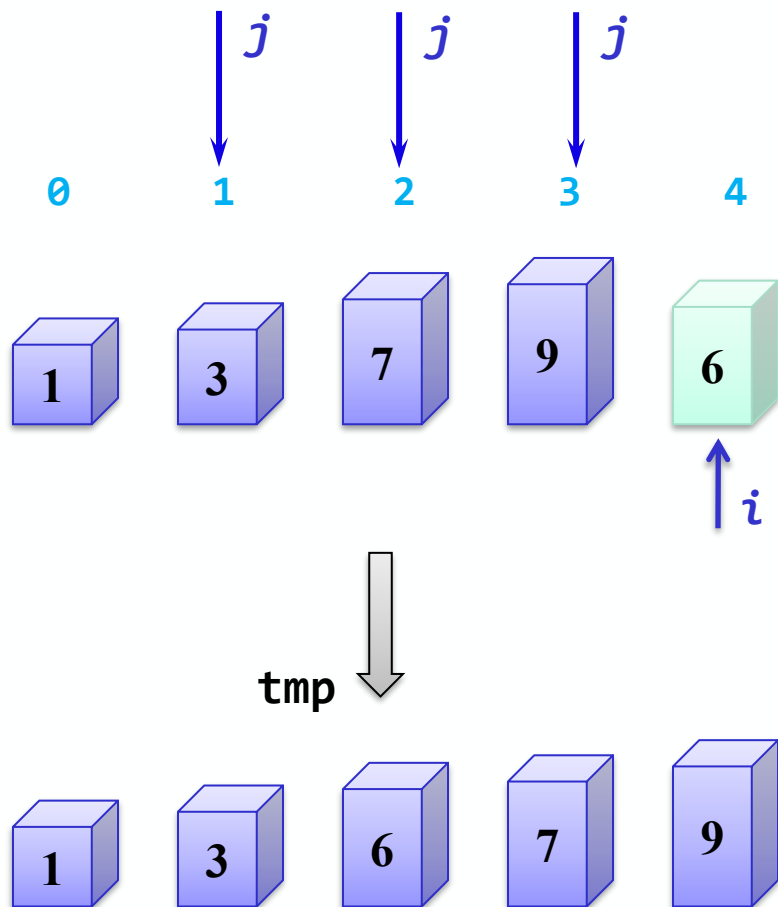
基本思想：序列中第1个关键字直接加入有序区，剩下的 $n-1$ 个关键字按顺序依次插入该有序区中，经过 $n-1$ 趟排序后即成为有序序列。（基于顺序查找定位插入位置）

假设 n 个关键字的序列， $R[0] \sim R[i-1]$ 已有序，在有序区中插入 $R[i]$



8.2.1 直接插入排序(P292)——临时变量法

一趟直接插入排序举例：



$R[i] \geq R[i-1]$ 无需插入
插入条件: $R[i] < R[i-1]$

😊 在有序区从后向前查找
 $\leq \text{tmp}$ 的第一个 $R[j]$,

$R[j+1] = \text{tmp}$

8.2.1 直接插入排序(P292)——临时变量法

排序过程：将一条记录插入到已排好的有序表中，从而得到一个新的、记录数量增1的有序表。

【例】已知待排序记录的关键字序列为 (13, 6, 3, 31, 9, 27, 5, 11)，请给出用直接插入排序法进行排序的过程。

【有序区】， 无序区

$R[i] \geq R[i-1]$ 无需插入

插入条件： $R[i] < R[i-1]$

😊 临时变量tmp存储记录 $R[i]$ 的值

$tmp = R[i]$

😊 $R[j] > tmp$ 时便向右移动 $R[j]$

$R[j+1] = R[j]$

😊 在有序区从后向前查找

$\leq tmp$ 的第一个 $R[j]$,

$R[j+1] = tmp$

$R[j]$ $R[i]$

【13】 , 6, 3, 31, 9, 27, 5, 11

【6, 13】 , 3, 31, 9, 27, 5, 11

【3, 6, 13】 , 31, 9, 27, 5, 11

【3, 6, 13, 31】 , 9, 27, 5, 11

【3, 6, 9, 13, 31】 , 27, 5, 11

【3, 6, 9, 13, 27, 31】 , 5, 11

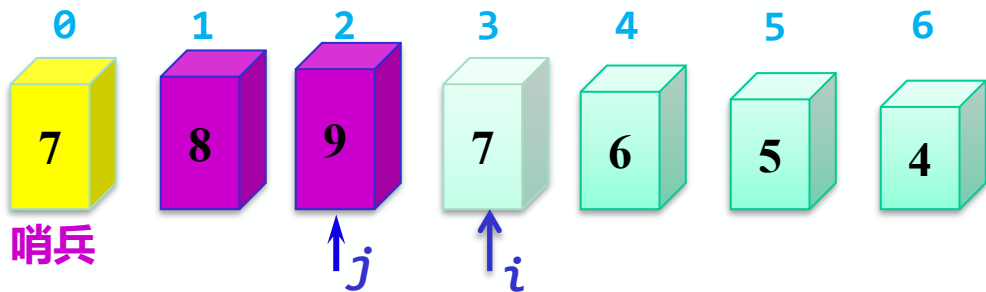
【3, 5, 6, 9, 13, 27, 31】 , 11

【3, 5, 6, 9, 11, 13, 27, 31】

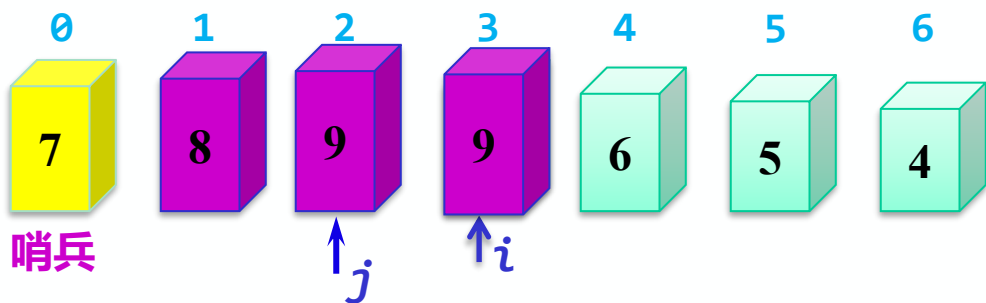
8.2.1 直接插入排序——哨兵法

【例】 设待排序的表有6个关键字，分别为 (9, 8, 7, 6, 5, 4)。说明采用直接插入排序方法进行排序的过程。

元素存在1-6号位置上，下标为0 ($R[0]$) 存放待排序元素 $R[i]$ (哨兵)

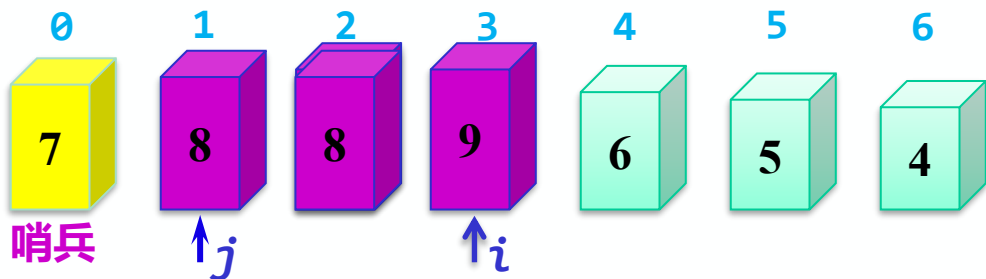


☺ $R[i] < R[i-1]$, $R[i]$ 复制给哨兵: $R[0] = R[i]$



☺ $R[j] >$ 哨兵, 右移 $R[j]$ 腾位

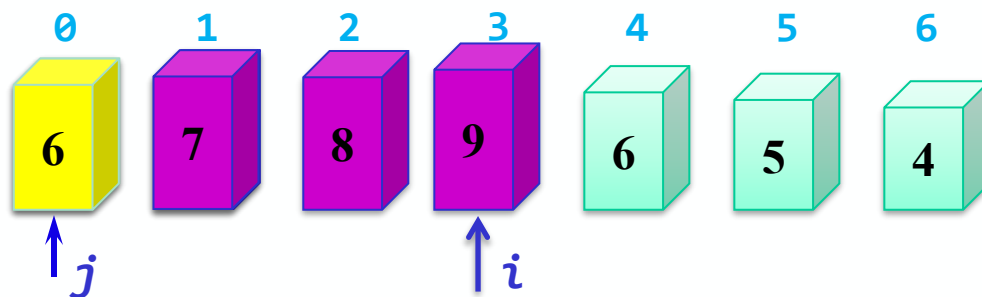
$R[j+1] = R[j]$
 $--j$



☺ $R[j] \leq$ 哨兵

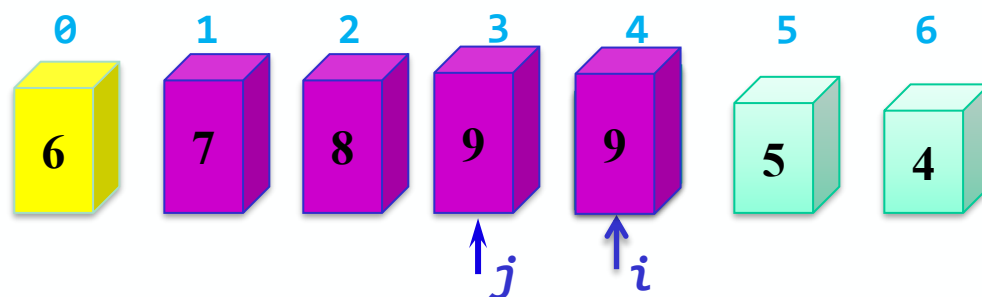
$R[j+1] = R[0]$

8.2.1 直接插入排序——哨兵法



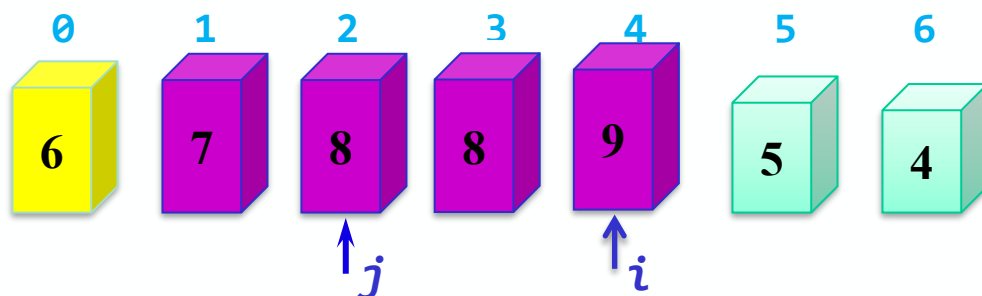
😊 $R[j] \leq \text{哨兵}$
 $R[j+1] = R[0]$

i 指向下一待排序记录



😊 $R[i] < R[i-1]$, $R[i]$ 复制
给哨兵: $R[0] = R[i]$

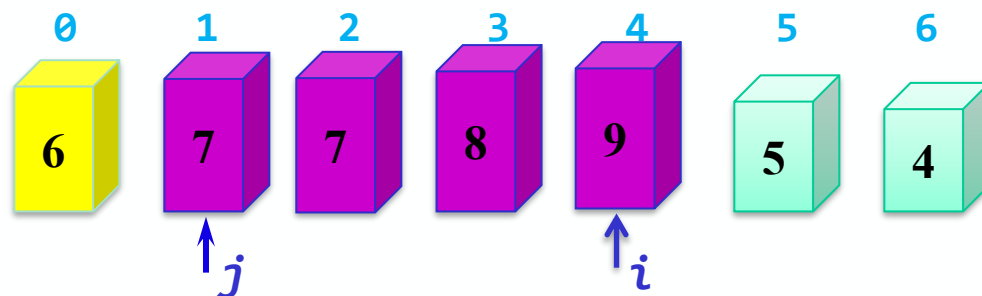
$j = i - 1$



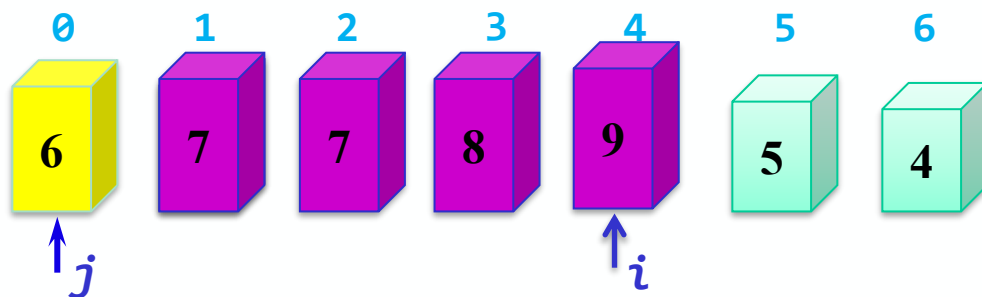
😊 $R[j] > \text{哨兵}$, 右移 $R[j]$ 腾位

$R[j+1] = R[j]$

$--j$



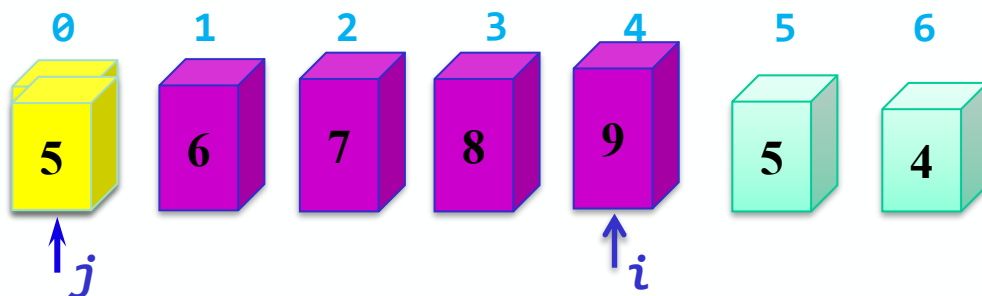
8.2.1 直接插入排序——哨兵法



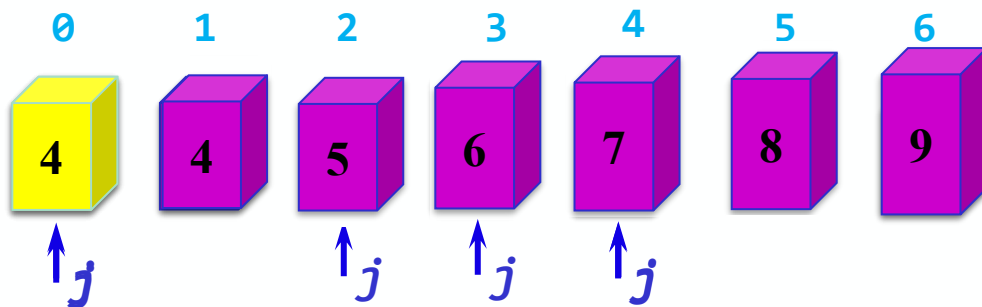
☺ $R[j] \leq \text{哨兵}$

$R[j+1] = R[0]$

i 指向下一待排序记录, $j = i - 1$



☺ $R[i] < R[i-1]$, $R[i]$ 复制
给哨兵: $R[0] = R[i]$



☺ $R[j] > \text{哨兵}$, 右移 $R[j]$ 腾位

$R[j+1] = R[j]$

-- j

完成!

直接插入排序的算法 (P292) :

```
void InsertSort(RedType R[], int n) { //在数组R上进行插入, n是元素个数
    int i, j; //指针i表示无序区待插入的第一个元素, j是寻找插入位置的下标
    for (i=2; i<=n; ++i) { //R[1]直接加入有序区, 依次将R[2]~R[n]插入有序区
        if (R[i]<R[i-1]) { //若<, 需将R[i]插入有序区
            R[0]=R[i]; //R[i]复制为哨兵, 用临时变量就是tmp=R[i];
            //找R[i]的插入位置
            for(j=i-1; R[j]>R[0]; --j){ //从后往前查找待插入位置
                R[j+1]=R[j]; //将关键字大于R[0]的记录右移挪位
            }
            R[j+1]=R[0]; //哨兵复制到插入位置R[j+1]
        }
    }
}
```

8.2.1 直接插入排序(P293)

算法分析

■ 最好的情况（关键字在记录序列中正序）：

如对（1,2,3,4,5）进行直接插入排序

1 2 3 4 5

累计比较4次（2 1, 3 2, 4 3, 5 4）

“比较”的次数：

$$\sum_{i=2}^n 1 = n - 1$$

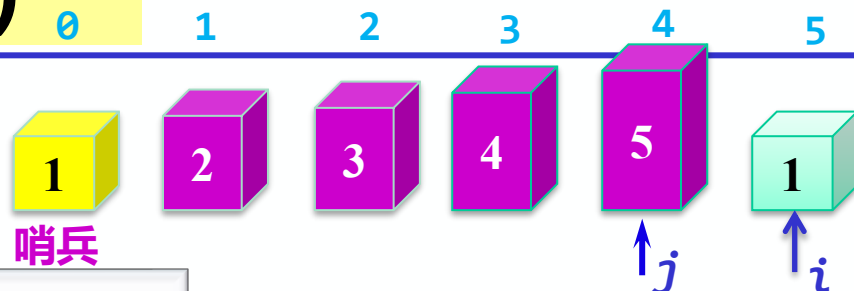
“移动”的次数：

0

最好的时间复杂度： $O(n)$

8.2.1 直接插入排序(P293)

算法分析



■ 最坏的情况（关键字在记录序列中反序）：

如 $(5, 4, 3, 2, 1)$ $R[i]$ 插入到最前面，比较 $i-1+1=i$ 次，移动 $i+1$ 次

“比较”的次数：

$$\sum_{i=2}^n i = \frac{(n+2)(n-1)}{2}$$

“移动”的次数：

$$\sum_{i=2}^n (i+1) = 3 + 4 + \cdots + n + 1 = \frac{(n+4)(n-1)}{2}$$

最坏情况的时间复杂度： $O(n^2)$

■ 平均情况下待排序表中元素是随机的，可以取上述最好与最坏情况的平均值作为平均情况下的时间复杂度，总的比较次数和总的移动次数均约为 $n^2/4$

平均情况的时间复杂度： $O(n^2)$ ，它是稳定的排序算法。

空间复杂度： $O(1)$ ，仅使用了常数个辅助单元。

8.2.1 直接插入排序(P293)

总结

最好的时间复杂度： $O(n)$

最坏情况的时间复杂度： $O(n^2)$

平均情况的时间复杂度： $O(n^2)$ ，它是**稳定**的排序算法。

空间复杂度： $O(1)$ ，仅使用了常数个辅助单元。

直接插入排序中所产生的有序区**不一定是全局有序的**。

每一趟排序并不一定能将一个元素放置在最终位置上（即不一定将一个元素归位）。

插入排序与待排序数据顺序有关，**正序效率高，反序效率低**。

8.2.1 直接插入排序

设有一组初始关键字序列为(24,35,12,27,18,26),
则第3趟直接插入排序结束后的结果是()。

第一趟： 24,35,12,27,18,26

第二趟： 24,35,12,27,18,26

第三趟： 12,24,35,27,18,26

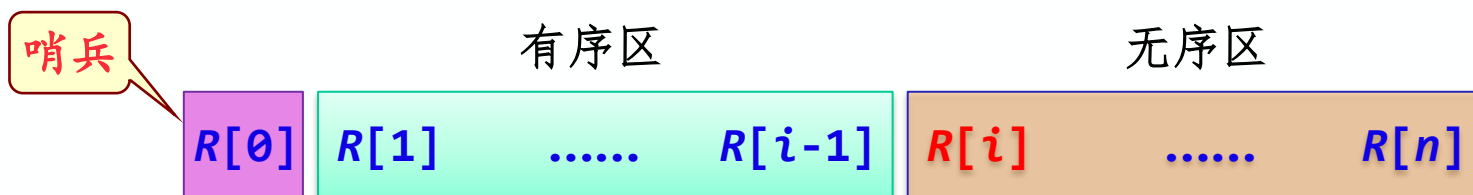
第四趟： 12,24,27,35,18,26

第五趟： 12,18,24,27,35,26

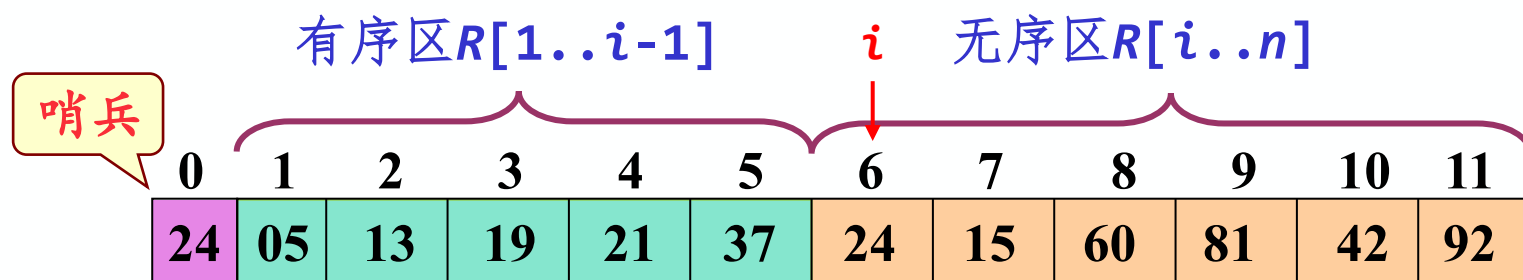
第六趟： 12,18,24,26,27,35

8.2.2 折半插入排序(P293)——哨兵法

基本思想：采用折半查找方法查找，称为**二分插入排序**或**折半插入排序**。



采用折半查找在有序区
找到插入的位置



$low=1$

$high$

$mid=(low+high)/2$

$low>high$ 查找失败！37右移挪位

$R[high+1]=R[0];$

//待排序关键字的插入位置是 $high+1$


```

void BInsertSort(RedType R[], int n){//在数组R上进行插入，n是元素个数
    int i,j,low,high,mid;
    for (i=2;i<=n;++i){
        if(R[i]<R[i-1]){
            R[0]=R[i];//当前插入元素R[i]存到“哨兵”位置
            low=1;high=i-1;//设置折半查找的范围
            while(low<=high){//有序区正序，采用折半查找法查找插入位置
                mid=(low+high)/2;//取中间点
                if(R[0].key<R[mid].key)    high=mid-1;//查找左半子表
                else if(R[0].key>R[mid].key) low=mid+1;//查找右半子表
                else break;
            }//循环结束，high+1则为插入位置
            for(j=i-1;j>=high+1;--j) //j右移比哨兵大的元素腾出位置
                R[j+1]=R[j];    //统一后移元素，空出插入位置
            R[high+1]=R[0];      //哨兵复制到插入位置R[high+1]
        }
    }
}

```

8.2.2 折半插入排序(P294)

算法分析

5	10	19	21	37	16
---	----	----	----	----	----

时间复杂度：折半插入排序**仅减少了关键字间的比较次数，而记录的移动次数不变。**因此折半插入排序的时间复杂度仍然为 $O(n^2)$

空间复杂度： $O(1)$ ，仅使用了常数个辅助单元，和直接插入排序相同。

平均性能优于直接插入排序，是一种**稳定**的排序方法。

8.2.3 希尔排序（缩小增量排序）（P295）

基本思想：先将待序列**分割成若干个形如** $L[i, i+d, i+2d, \dots, i+kd]$ **的子序列**，分别进行**直接插入排序**，当整个序列中的记录呈“**基本有序**”时，再对**全体记录进行一次直接插入排序**。

基本思路：

- ① $d = \lfloor n/2 \rfloor$
- ② 将排序序列分为 d 个组，在各组内进行**直接插入排序**
- ③ 递减 $d = \lfloor d/2 \rfloor$ ，重复②，直到 $d=1$



算法最后一趟对所有数据进行了直接插入排序，所以结果一定是正确的。

8.2.3 希尔排序 (缩小增量排序) (P295)

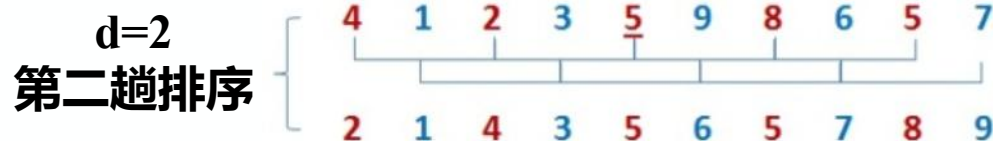
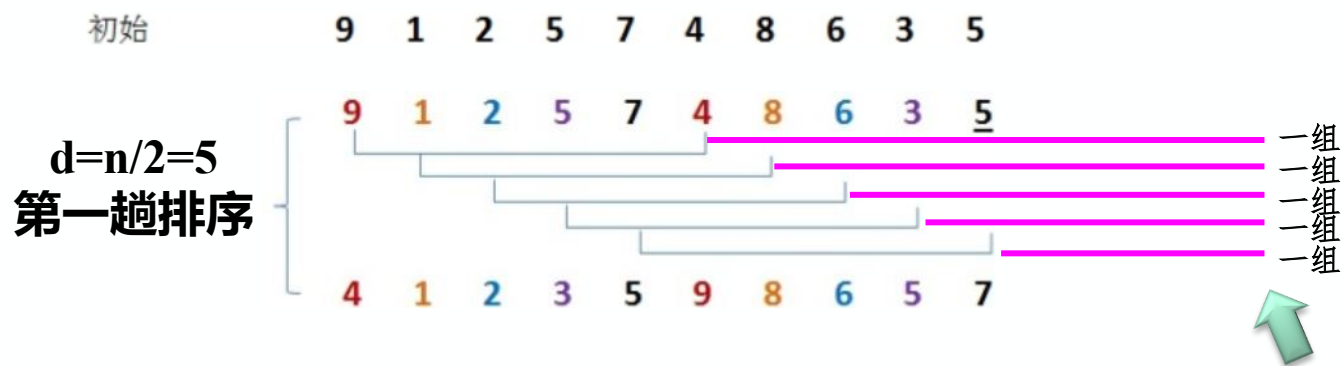
	①	②	③	④	⑤	⑥	⑦	⑧	⑨	⑩
初始序列	9	8	7	6	5	4	3	2	1	0
$d=5$	9	8	7	6	5	4	3	2	1	0
直接插入排序	4	3	2	1	0	9	8	7	6	5
$d=d/2=2$	4	3	2	1	0	9	8	7	6	5
直接插入排序	0	1	2	3	4	5	6	7	8	9
$d=d/2=1$	0	1	2	3	4	5	6	7	8	9
直接插入排序	0	1	2	3	4	5	6	7	8	9

注意：对于最后一趟 $d=1$ ，排序前的数据已将近正序！

8.2.3 希尔排序(P296)

- 希尔排序的组内排序采用的是直接插入排序
- 待整个序列中记录“基本有序”时，再对全体记录进行1次直接插入排序

例：将 n 个记录序列分成 d 个子序列，分别对每个子序列进行直接插入排序。



排序完成 🍵

稳定吗？

【例】关键字序列 $T=(49, 38, 65, 97, 76, 13, 27, 49^*, 55, 04)$
请写出希尔排序的具体实现过程，增量取 $(5, 3, 1)$

$r[i]$	0	1	2	3	4	5	6	7	8	9	10
初态:		49	38	65	97	76	13	27	49*	55	04
第1趟 ($d=5$)		13	27	49*	55	04	49	38	65	97	76
第2趟 ($d=3$)		13	04	49*	38	27	49	55	65	97	76
第3趟 ($d=1$)		04	13	27	38	49*	49	55	65	76	97

- (1) 第一趟取增量 $d=5$ ，所有间隔为5的记录为一组，全部记录分成5组，在各个组中分别直接插入排序。
- (2) 第二趟取增量 $d=3$ ，所有间隔为3的记录为一组，全部记录分成3组，在各个组中分别直接插入排序。
- (3) 第三趟取增量 $d=1$ ，对整个序列进行一趟直接插入排序，排序完成。

8.2.3 希尔排序(P296)

算法分析

■ 时间复杂度

由于希尔排序的时间复杂度依赖于增量序列的函数，这是数学上尚未解决的难题。所以其时间复杂度分析比较困难。当 n 在某个特定范围时，**希尔排序的时间复杂度约为 $O(n^{1.3})$** ，**最好时间复杂度为 $O(n)$** ，**最坏时间复杂度为 $O(n^2)$** 。

■ 稳定性

当相同关键字的记录被划分到不同的子表时，可能会改变它们之间的相对次序，因此，**希尔排序**是一个**不稳定的**排序方法。例如上例中49和49*。显然，49和49*的相对次序已经发生了变化。

■ 空间复杂度： $O(1)$

8.2.3 希尔排序(P296)

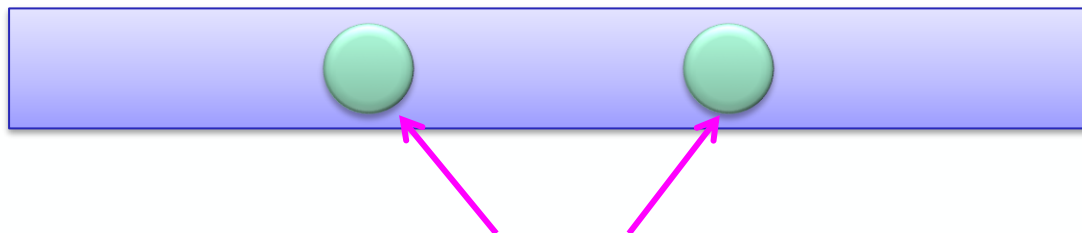
总结

希尔排序的每一趟并不产生有序区，也不一定将一个元素放置在最终位置上（即不一定将一个元素归位）。

8.3 快速排序（交换排序）（P297）



基本思想：两两比较，如果发生逆序则交换，直到所有记录都排好序为止。



两个元素逆序时进行交换

常见的交换排序方法：

- （1）冒泡排序（或起泡排序）
- （2）快速排序

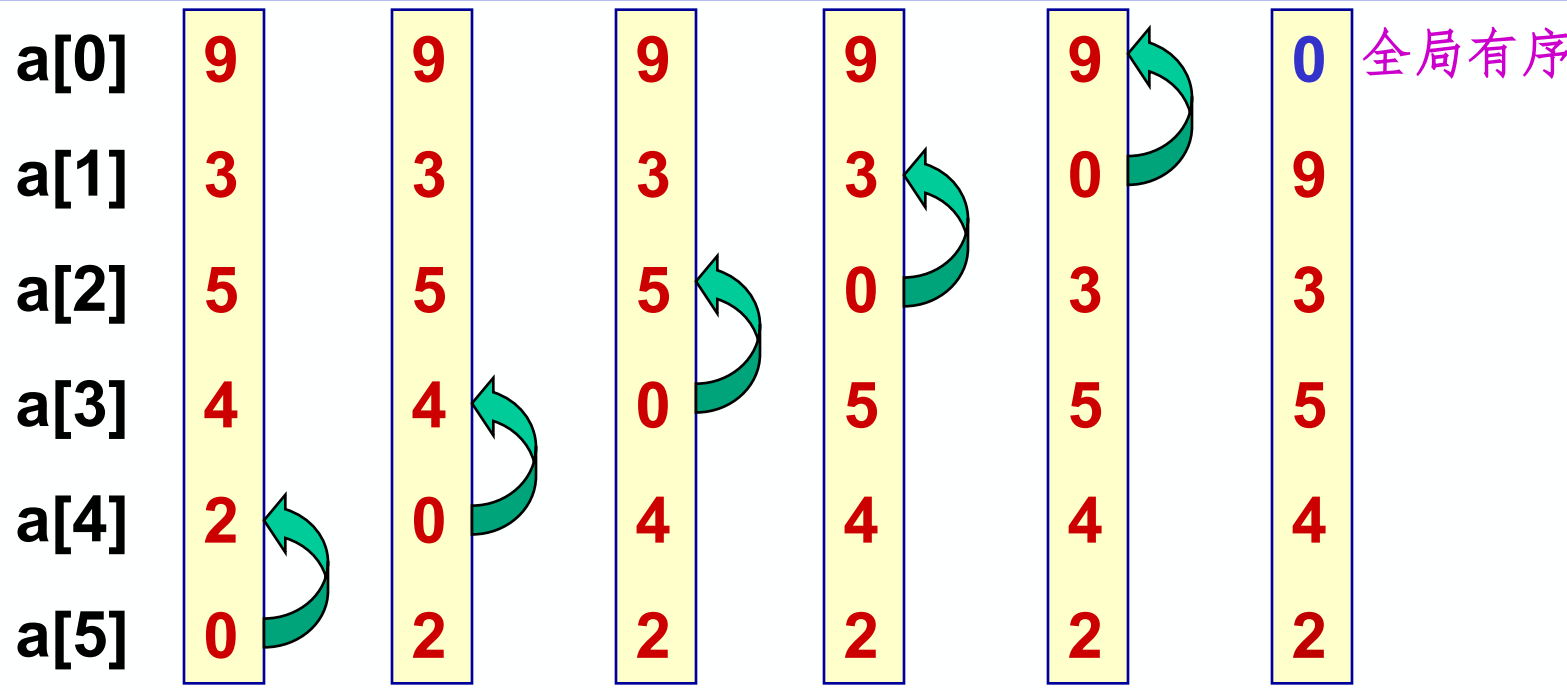
8.3.1 冒泡排序 (P297)

基本思想：从后往前（或从前往后）**两两比较相邻元素的值**，若为**逆序**（即 $R[i-1] > R[i]$ ）则**交换**它们，直至序列比较完毕，一趟冒泡后，将**最小的元素**交换到待排序列的第一个位置。下一趟冒泡时，前一趟确定的最小元素不再参与比较。

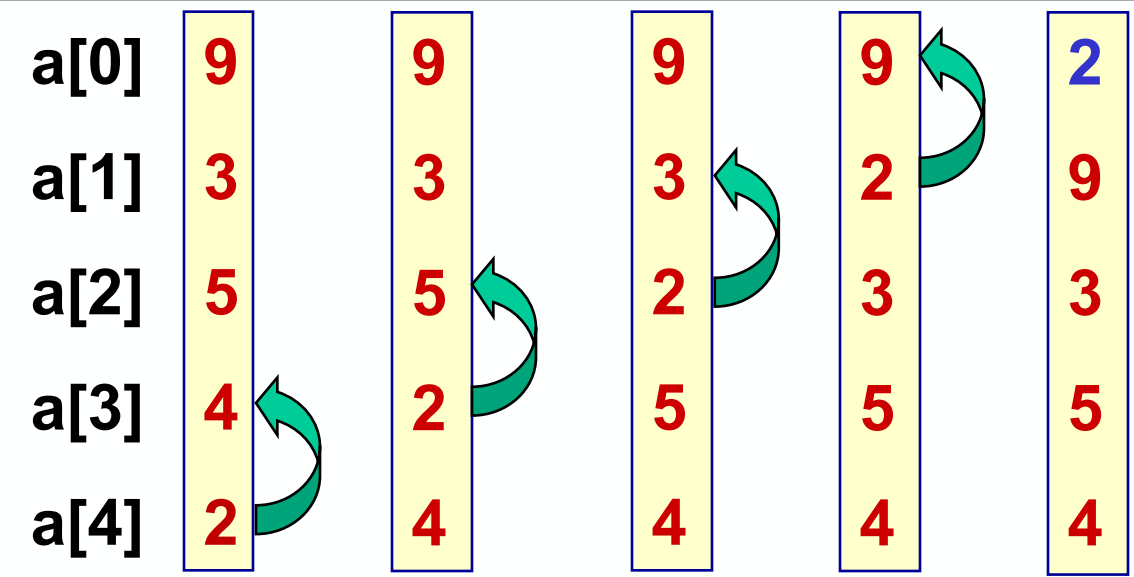
**有序区总是
全局有序的**

【例】用冒泡法对六个数：9，3，5，4，2，0进行排序（由小到大）

第一趟：

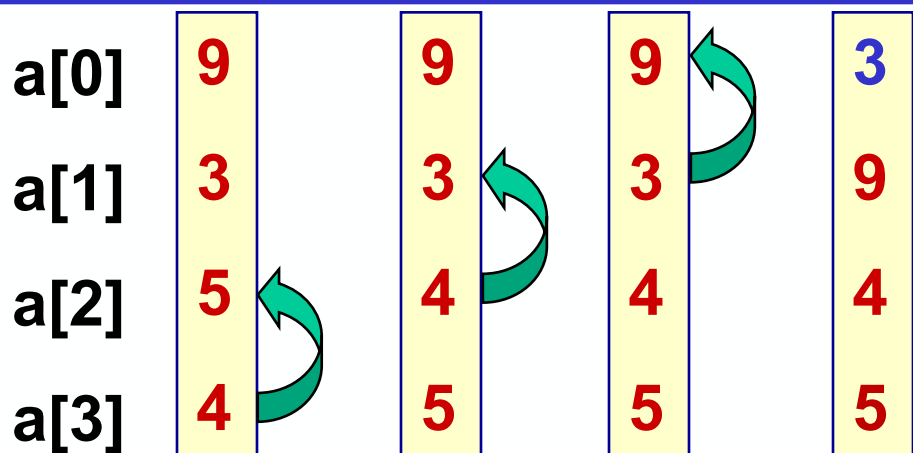


第二趟：

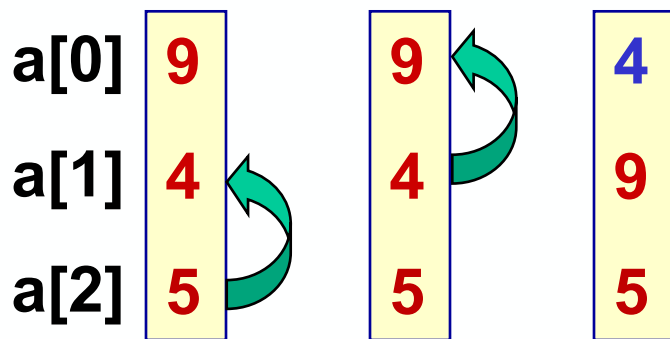


冒泡排序在每趟排序后，都会有一个元素被放置到其最终位置上。

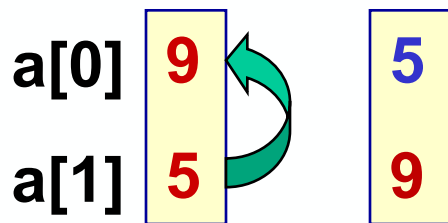
第三趟:



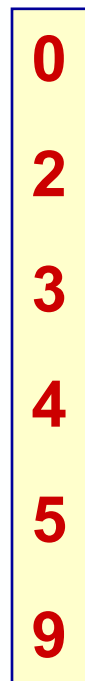
第四趟:



第五趟:



最终结果:



【例】 设待排序序列为{12, 2, 16, 30, 28, 10, 16*, 20, 6, 18}。
写出使用冒泡排序，每趟排序结束后关键字序列的状态。

冒泡排序

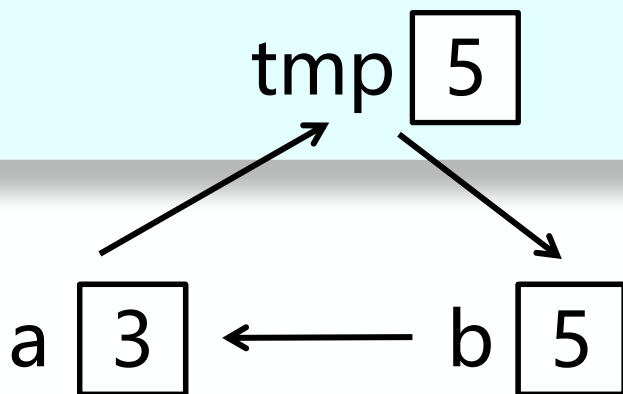
第一趟:	[2]	12	6	16	30	28	10	16*	20	18
第二趟:	[2	6]	12	10	16	30	28	16*	18	20
第三趟:	[2	6	10]	12	16	16*	30	28	18	20
第四趟:	[2	6	10	12]	16	16*	18	30	28	20
第五趟:	[2	6	10	12	16]	16*	18	20	30	28
第六趟:	[2	6	10	12	16	16*]	18	20	28	30
第七趟:	[2	6	10	12	16	16*	18]	20	28	30
第八趟:	[2	6	10	12	16	16*	18	20]	28	30
第九趟:	[2	6	10	12	16	16*	18	20	28]	30

一旦某一趟比较时不出现记录交换，说明已排好序了，可以结束

冒泡排序的算法:

```
void BubbleSort(RedType R[], int n) { //在数组R上进行排序, n是元素个数
    for(int i=0; i<n-1; i++){ //表示本趟冒泡是否发生交换的标志
        flag=false; //默认未交换
        for(int j=n-1; j>i; j--){ //从后往前一趟冒泡, 将最小的移动到最前面
            if(R[j-1].key>R[j].key){ //若为逆序
                swap(R[j-1], R[j]); //交换, 3次移动
                flag = true; //本轮发生交换
            }
        }
        if(flag == false){
            break; //本趟遍历没有发生交换, 说明表已有序, 中途结束算法
        }
    }
}
```

```
void swap(int &a, int &b) {
    int tmp=a;
    a=b;
    b=tmp;
}
```



比较发现逆序,
交换要三次移动

冒泡排序(P298)

算法分析

五个元素只需比较四次

■ 最好的情况（关键字在记录序列中正序）：只需进行一趟冒泡

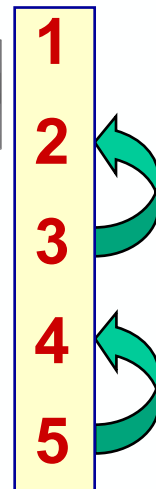
“比较”次数：

$n-1$

“移动”的次数：

0

最好时间复杂度： $O(n)$



■ 最坏的情况（关键字在记录序列中反序）：第*i*趟需进行*n-i*次比较

“比较”次数=交换次数：

$$(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

“移动”的次数：

$$3 * \frac{n(n-1)}{2}$$

最坏时间复杂度： $O(n^2)$

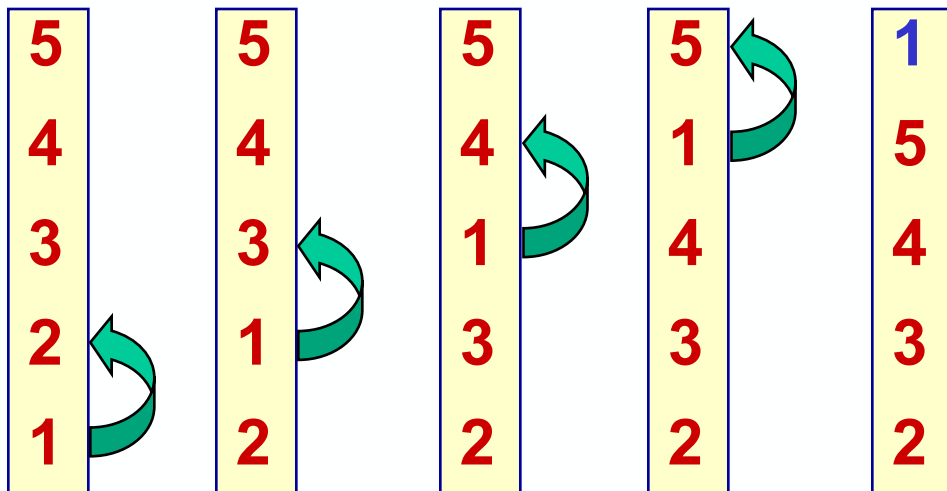
每次交换都需要
移动元素3次

```
//交换  
void swap(int &a, int &b){  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

冒泡排序(P297)

第一趟:

$i=1$

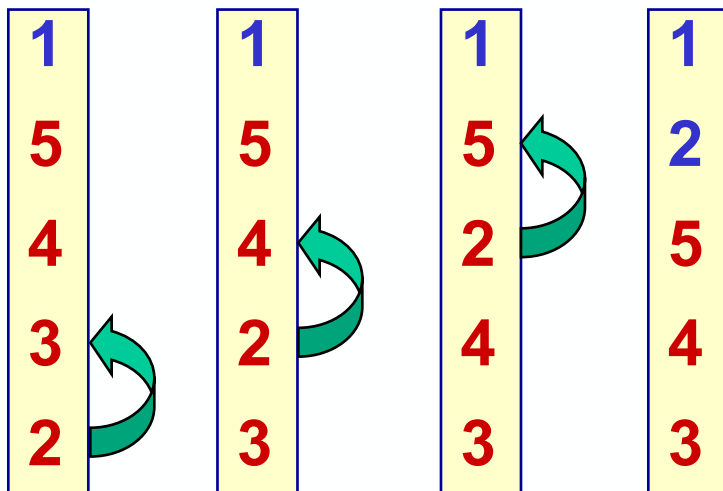


比较的次数=4
 $=5-1$

移动的次数=3*4
 $=12$

第二趟:

$i=2$



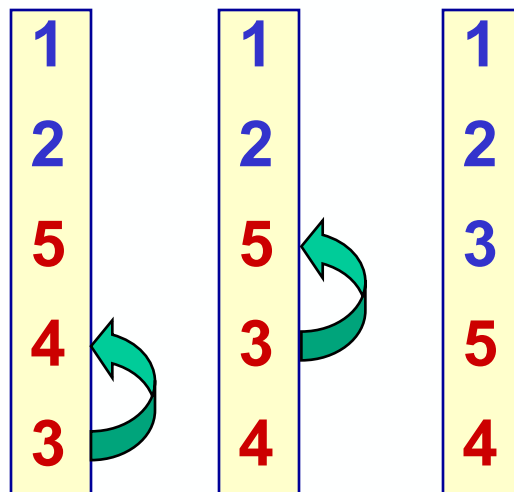
比较的次数=3
 $=5-2$

移动的次数=3*3
 $=9$

冒泡排序(P297)

第三趟:

$i=3$

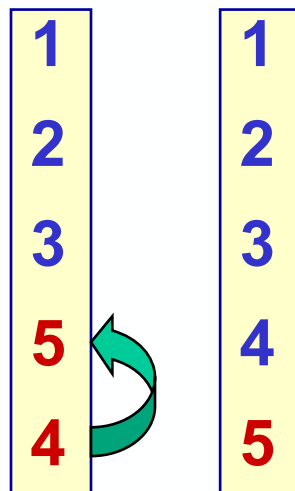


比较的次数=2
 $=5-3$

移动的次数= $3*2$
 $=6$

第四趟:

$i=4$



比较的次数=1
 $=5-4$

移动的次数= $3*1$
 $=3$

冒泡排序(P298)

总结

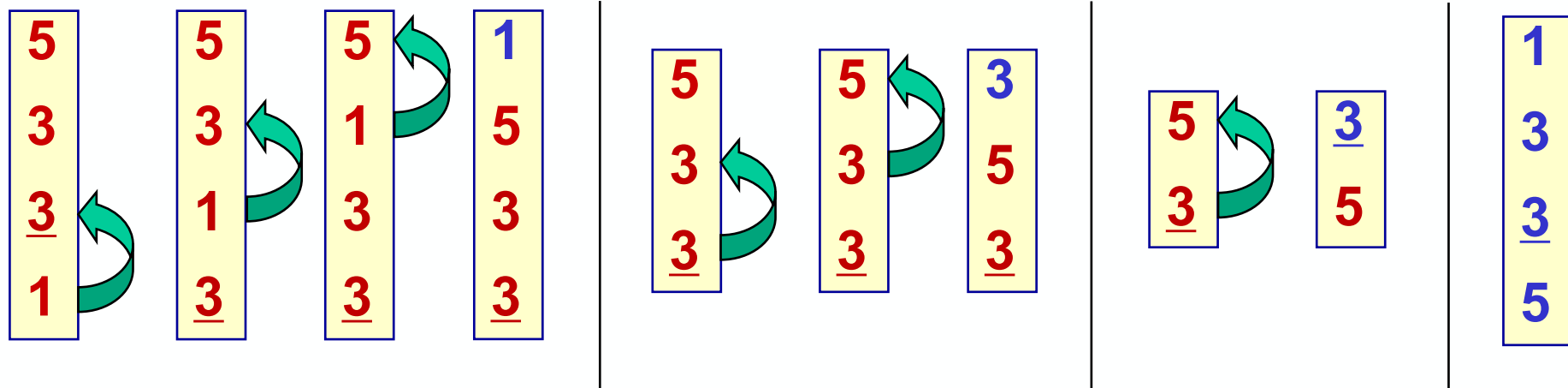
最好的时间复杂度： $O(n)$

最坏情况的时间复杂度： $O(n^2)$

平均情况的时间复杂度： $O(n^2)$ ，它是**稳定**的排序算法。

空间复杂度： $O(1)$ ，仅使用了常数个辅助单元。（辅助变量*i*，*j*、*tmp*）

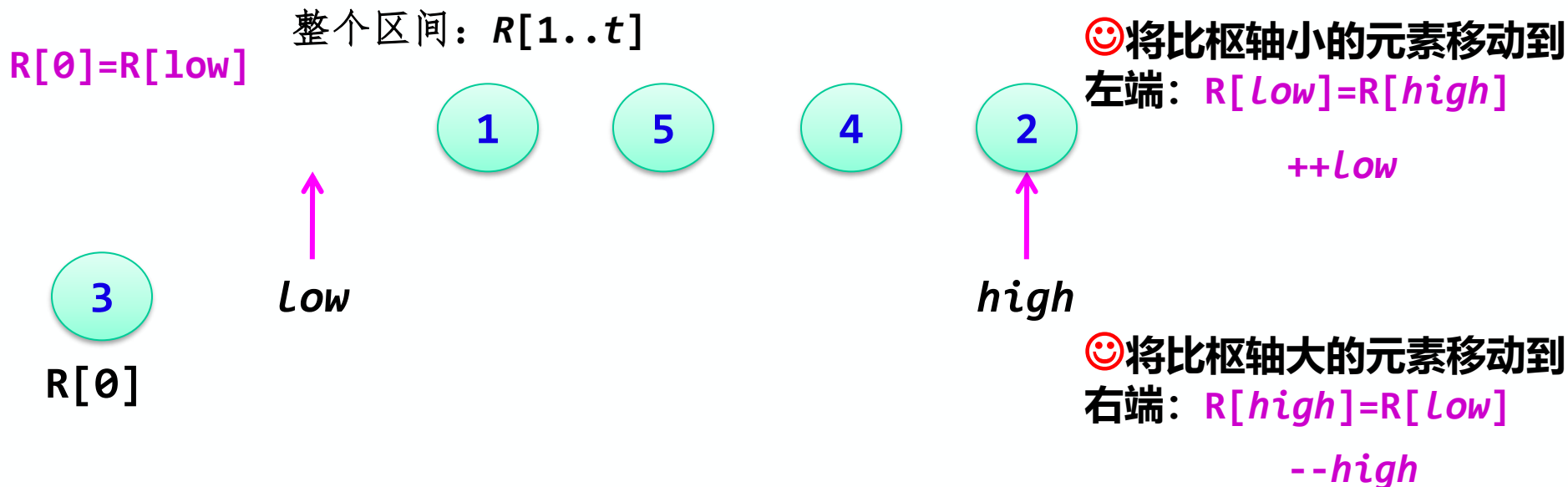
冒泡排序中所产生的**有序子序列一定是全局有序的**。



8.3.2 快速排序 (P298)

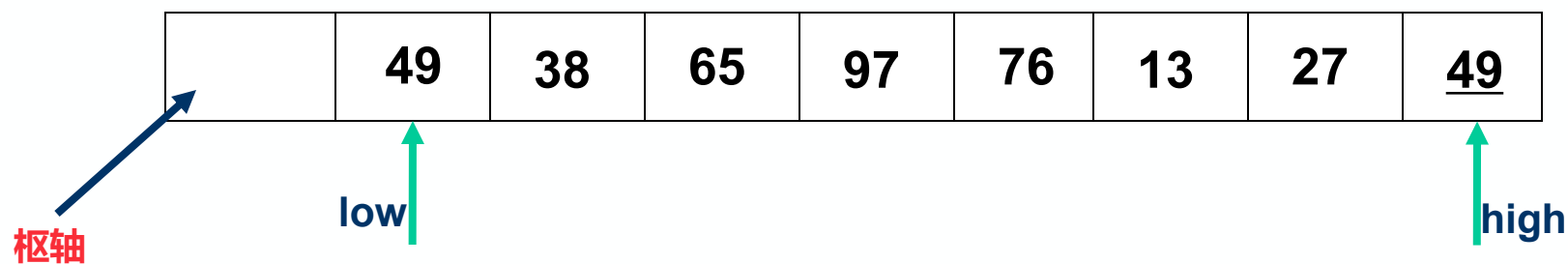
基本思想：①任取一个元素（第一个）为**中心**；
②所有比它**小**的元素一律**左放**，比它**大**的元素一律**右放**，形成左右两个子表，当 $low==high$ ：把pivot放在其最终位置上；
③递归地对两个子表重新选择枢轴，并重复上述过程，**直至每个子表内只有一个元素或为空为止**，那么所有元素就都放在了其最终位置上。

枢轴 (pivot)



快速排序

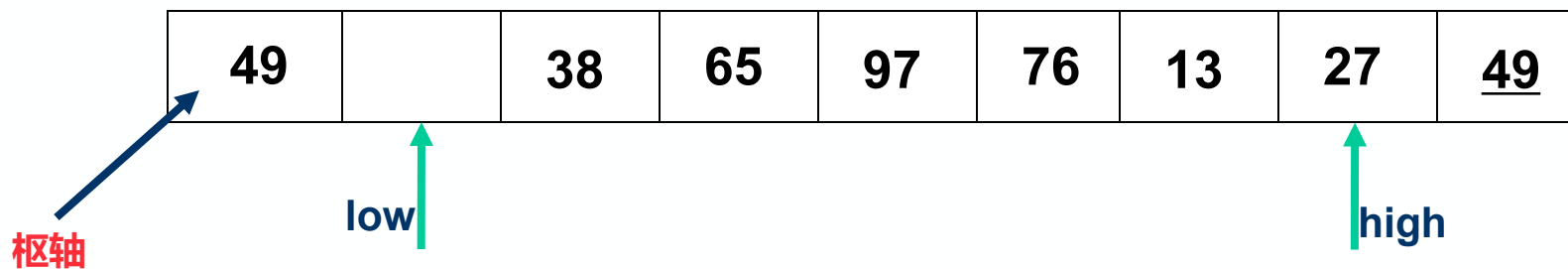
0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>



- (1) 选择待排序的第一个记录作为枢轴/支点，附设两指针 $low=1$ ， $high=length$ ，指针 low 指向的49先暂存在 $R[0]$ 的位置。同时：
- (2) $high$ 从表的右端依次向左搜索，比较 $high$ 所指元素和枢轴的值，如果 $high$ 的值比枢轴小，则将值赋到 low 位置，否则 $--high$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>



low指向位置空出，再通过high依次向左搜索：

若high所指元素比枢轴小， $R[low]=R[high]$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	65	97	76	13		<u>49</u>
----	----	----	----	----	----	----	--	-----------

枢轴

low

high

high指向位置空出，再通过low指针依次向右搜索：

若low所指元素比枢轴小，则++low；若low所指元素比枢轴大，

$R[high]=R[low]$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	65	97	76	13		<u>49</u>
----	----	----	----	----	----	----	--	-----------

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	65	97	76	13		<u>49</u>
----	----	----	----	----	----	----	--	-----------

枢轴

low

high

low所指元素比枢轴大, $R[high]=R[low]$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38		97	76	13	65	<u>49</u>
----	----	----	--	----	----	----	----	-----------

枢轴

low

high

low指向位置空出，再通过high指针依次向左搜索：
若high所指元素比枢轴大，则--high；

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38		97	76	13	65	<u>49</u>
----	----	----	--	----	----	----	----	-----------

枢轴

low

high

low指向位置空出，再通过high指针依次向左搜索：

若high所指元素比枢轴小， $R[low]=R[high]$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	13	97	76		65	<u>49</u>
----	----	----	----	----	----	--	----	-----------

枢轴

low

high

high指向位置空出，再通过low指针依次向右搜索：

若low所指元素比枢轴小，则++low；若low所指元素比枢轴大，

$R[high]=R[low]$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	13	97	76		65	<u>49</u>
----	----	----	----	----	----	--	----	-----------

枢轴

low

high

high指向位置空出，再通过low指针依次向右搜索：

若low所指元素比枢轴小，则++low；若low所指元素比枢轴大，

$R[high]=R[low]$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	13		76	97	65	<u>49</u>
----	----	----	----	--	----	----	----	-----------

枢轴

low

high

low指向位置空出，再通过high指针依次向左搜索：
若high所指元素比枢轴大，则--high；

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	13		76	97	65	<u>49</u>
----	----	----	----	--	----	----	----	-----------

枢轴

low

high

low指向位置空出，再通过high指针依次向左搜索：
若high所指元素比枢轴大，则--high；

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	13		76	97	65	<u>49</u>
----	----	----	----	--	----	----	----	-----------

low high

枢轴

直到 $low == high$ ，区间处理完毕，将枢轴元素放在该位置：
 $R[low] = R[0]$

快速排序

0	1	2	3	4	5	6	7	8
	49	38	65	97	76	13	27	<u>49</u>

49	27	38	13	49	76	97	65	<u>49</u>
----	----	----	----	----	----	----	----	-----------

low high

枢轴

第1趟排序结果

直到 $low == high$ ，区间处理完毕，将枢轴元素放在该位置：
 $R[low] = R[0]$

快速排序

0	1	2	3	4	5	6	7	8
	27	38	13	49	76	97	65	<u>49</u>

	27	38	13	49	76	97	65	<u>49</u>
--	----	----	----	----	----	----	----	-----------

枢轴

low

high

49左边的元素都是比它小的
49右边的元素都是比它大的
再递归地对左半子表和右半子表快速排序...

快速排序

0	1	2	3	4	5	6	7	8
	27	38	13	49	76	97	65	<u>49</u>

27	13	38		49	76	97	65	<u>49</u>
----	----	----	--	----	----	----	----	-----------

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	27	38	13	49	76	97	65	<u>49</u>

27	13	38		49	76	97	65	<u>49</u>
----	----	----	--	----	----	----	----	-----------

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	27	38	13	49	76	97	65	<u>49</u>

27	13		38	49	76	97	65	<u>49</u>
----	----	--	----	----	----	----	----	-----------

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	27	38	13	49	76	97	65	<u>49</u>

27	13		38	49	76	97	65	<u>49</u>
----	----	--	----	----	----	----	----	-----------

枢轴

low ↑
high ↑

直到 $low == high$ ，将枢轴元素放在该位置： $R[low] = R[0]$

左半子表的两个子表只有一个元素，左半子表处理完毕

第2趟排序结束

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

	13	27	38	49	76	97	65	<u>49</u>
--	----	----	----	----	----	----	----	-----------

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

76	13	27	38	49	<u>49</u>	97	65	
----	----	----	----	----	-----------	----	----	--

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

76	13	27	38	49	<u>49</u>	97	65	
----	----	----	----	----	-----------	----	----	--

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

76	13	27	38	49	<u>49</u>		65	97
----	----	----	----	----	-----------	--	----	----

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

76	13	27	38	49	<u>49</u>		65	97
----	----	----	----	----	-----------	--	----	----

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

76	13	27	38	49	<u>49</u>	65		97
----	----	----	----	----	-----------	----	--	----

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

76	13	27	38	49	<u>49</u>	65		97
----	----	----	----	----	-----------	----	--	----

枢轴

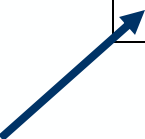
low high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

	13	27	38	49	<u>49</u>	65	76	97
--	----	----	----	----	-----------	----	----	----

枢轴



low high



第3趟排序结束

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	76	97	65	<u>49</u>

	13	27	38	49	<u>49</u>	65	76	97
--	----	----	----	----	-----------	----	----	----

枢轴

low high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	<u>49</u>	65	76	97

	13	27	38	49	<u>49</u>	65	76	97
--	----	----	----	----	-----------	----	----	----

枢轴

low

high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	<u>49</u>	65	76	97

<u>49</u>	13	27	38	49		65	76	97
-----------	----	----	----	----	--	----	----	----

枢轴

low high

快速排序

0	1	2	3	4	5	6	7	8
	13	27	38	49	<u>49</u>	65	76	97

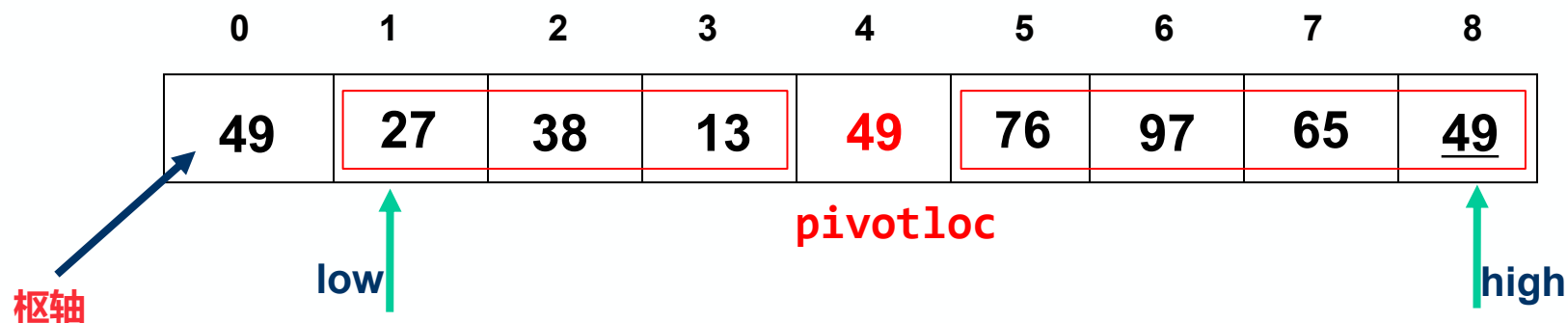
	13	27	38	49	<u>49</u>	65	76	97
--	----	----	----	----	-----------	----	----	----

枢轴

low high

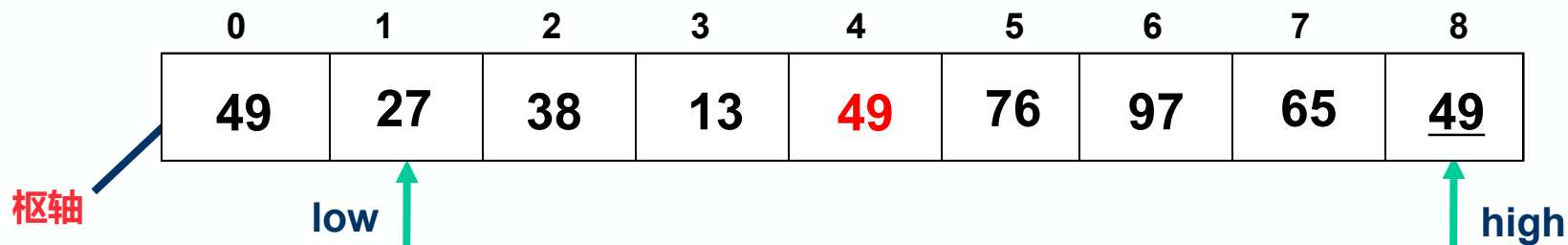
第4趟排序，全部结束

快速排序的算法 (P300) :



```
void QuickSort(RedType R[],int low,int high){//快速排序递归函数
    if (low<high){ //长度大于1, 直到low==high, 就是本趟枢轴的最终位置
        //Partition()就是划分操作, 将表R[low...high]划分为左右子表
        int pivotloc=Partition(R,low,high);//pivotlocation枢轴的位置
        QuickSort(R,low,pivotloc-1); //对左半子表递归快速排序
        QuickSort(R,pivotloc+1,high); //对右半子表递归快速排序
    }
}
```

快速排序的算法 (P300) :



```
int Partition(RedType R[],int low,int high){ //一趟快速排序过程
    R[0]=R[low]; //将当前表中第一个元素设为枢轴值
    while (low<high){ //从两端交替向中间扫描,直至low=high为止
        while ((low<high)&&R[high]>=R[0]) //low所指位置空
            --high; //high指针从右向左扫描
        R[low]=R[high]; //将比枢轴值小的元素移动到R[low]处
        while ((low<high)&&R[low]<=R[0]) //high所指位置空
            ++low; //low指针从左向右扫描
        R[high]=R[low]; //将比枢轴值大的元素移动到R[high]处
    }
    R[low]=R[0]; //枢轴元素存放到最后位置
    return low; //返回存放枢轴的最后位置
}
```

一组记录关键字为{50, 80, 56, 29, 35, 84}, 利用快速排序的方法进行第一趟排序后的结果为 ()。

A. 29, 35, 56, 80, 50, 84

B. 35, 29, 50, 56, 80, 84

C. 35, 29, 56, 50, 80, 84

D. 29, 35, 50, 56, 80, 84

答案: B

下列选项中，不可能是快速排序第2趟排序结果的是
()。

A、 2,3,5,4,6,7,9

B、 2,7,5,6,4,3,9

C、 3,2,5,4,7,6,9

D、 4,2,3,5,7,6,9

答案： C

解释：一趟快速排序会确定一个元素的最终位置，比它小的都在左边，比它大的都在右边。C只确定了9的最终位置

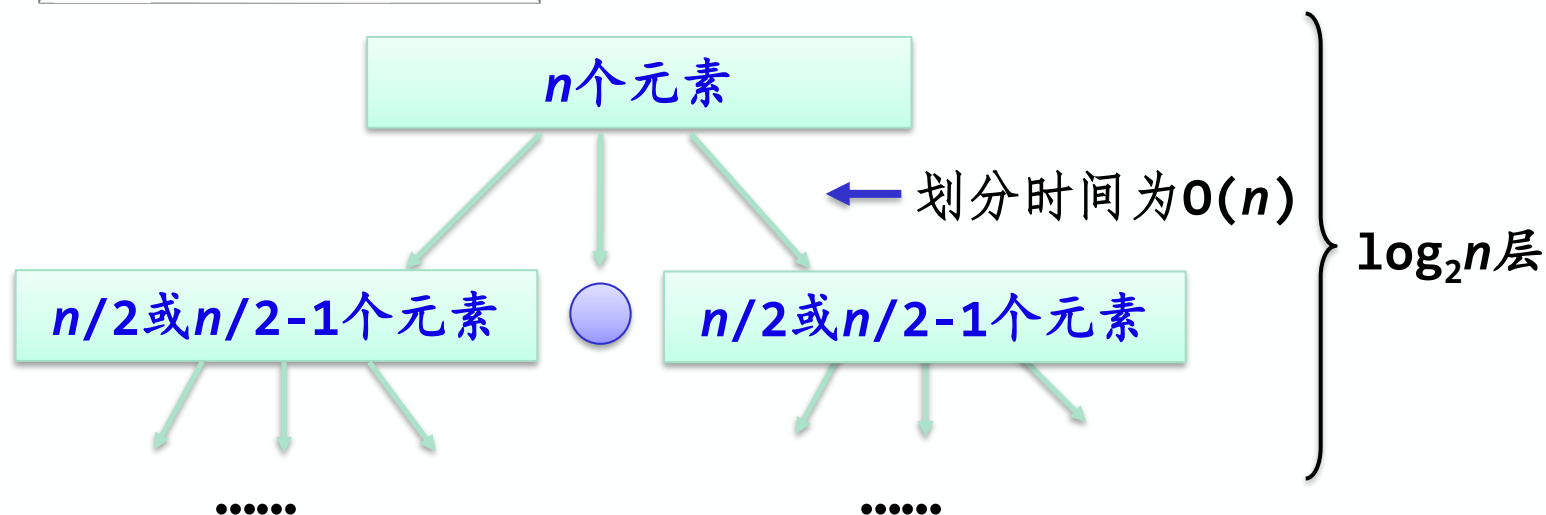
8.3 快速排序(P300)

算法分析

■ 最好情况：

Qsort(): $O(\log_2 n)$

确定枢轴位置Partition(): $O(n)$



每一趟排序后都能将待排序区间均匀分割成两个长度大致相等的子表

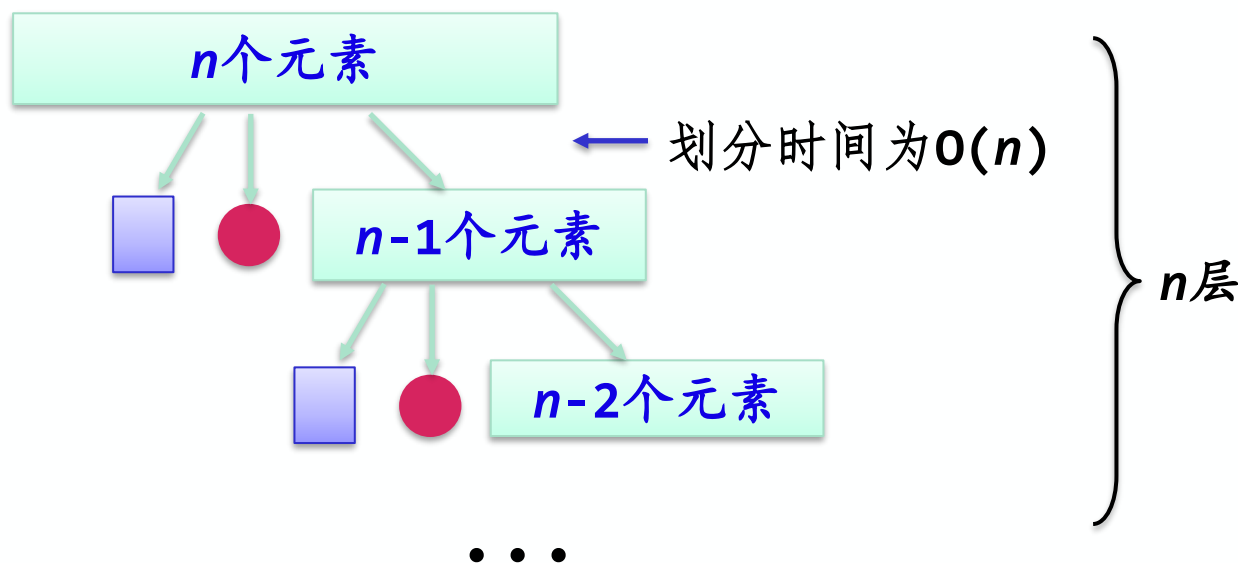
此时的时间复杂度为 $O(n\log_2 n)$ ，空间复杂度为 $O(\log_2 n)$ 。

8.3 快速排序(P300)

算法分析

■ 最坏情况:

Qsort(): $O(n)$
确定枢轴位置Partition(): $O(n)$



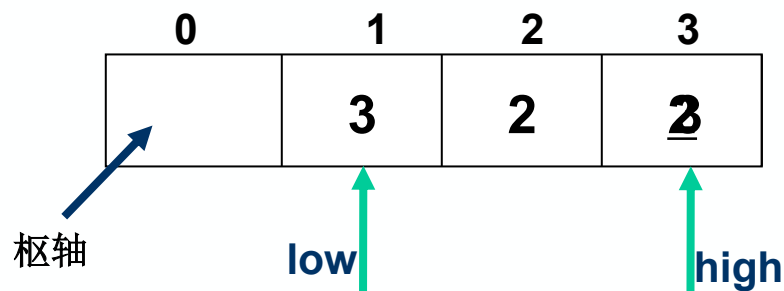
每次划分只能得到比上一次少一个元素的子区间，一趟只对一个元素排序
此时的时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

8.3 快速排序(P300)

算法分析

■ 稳定性

在划分算法中，若右半子表存在两个关键字相同，且均小于枢轴值，则在交换到左半子表后，它们的相对位置发生变化。因此，**快速排序**是一个**不稳定**的排序方法。例如下例中，2和2的相对位置已经发生了变化。



8.3 快速排序(P300)

总结

最好时间复杂度: $O(n\log_2 n)$

最坏时间复杂度: $O(n^2)$

平均时间复杂度: $O(n\log_2 n)$

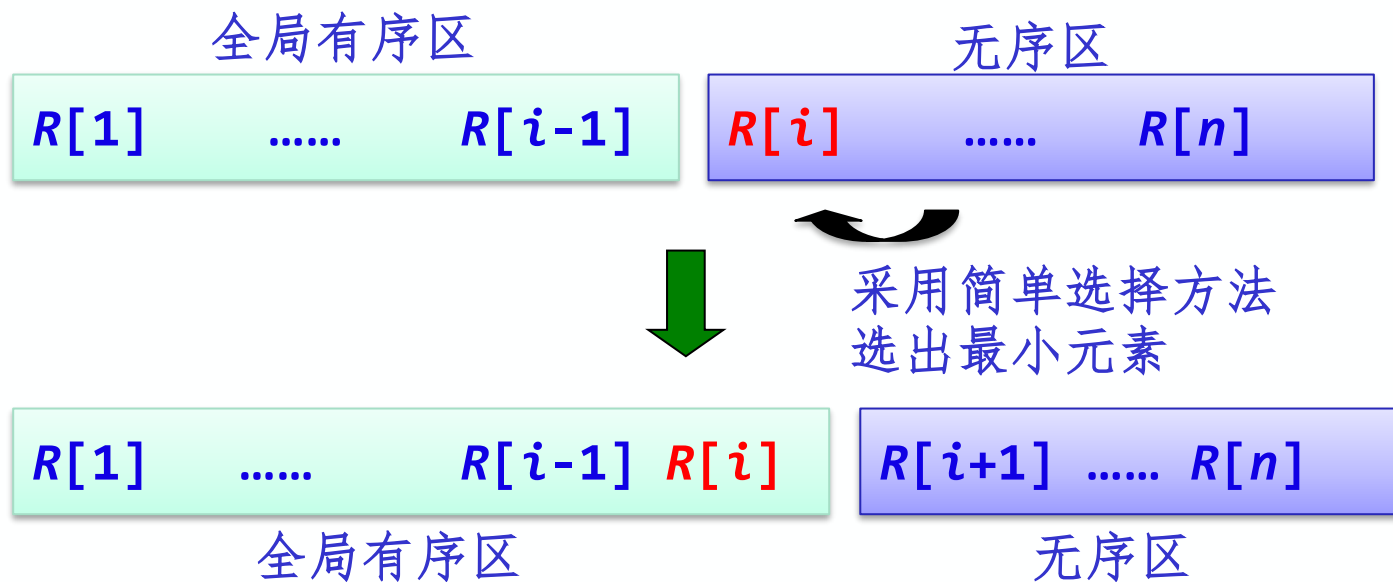
最好情况的空间复杂度: $O(\log_2 n)$ 最坏情况的空间复杂度: $O(n)$

快速排序不稳定，不是每趟都产生有序区，但它将枢轴放置到其最终位置上。快速排序与待排序数据的顺序有关，数据越接近有序，快排效率越低；数据越随机分布，快速排序的性能越好、效率越高。排序需要定位上界low和下界high的位置，所以适用于顺序结构，链式结构不适用。

8.4 选择排序(P301)



基本思想：每次从当前待排序的记录中选取**关键字最小**的记录，然后与待排序的记录序列中的第一个记录进行交换，直到整个记录序列有序为止。



■ 初始时，全局有序区为空； $i=1\sim n-1$ ，共经过 $n-1$ 趟排序

【例】初始记录的关键字7 4 -2 19 13 6，请写出简单选择排序法的过程。

初始序列: 7 4 -2 19 13 6

比较的次数=5
=6-1
=n-i

标记第一个元素 向后遍历, 找到小于标记元素的最小元素

第一趟排序 i=1: -2 4 7 19 13 6

比较的次数=4
=6-2

标记第一个元素

第二趟排序 i=2: -2 4 7 19 13 6

比较的次数=3
=6-3

标记第一个元素 向后遍历, 找到小于标记元素的最小元素

第三趟排序 i=3: -2 4 6 19 13 7

第四趟排序 i=4: -2 4 6 7 13 19

第五趟排序 i=5: -2 4 6 7 13 19

六个元素五趟排序实现。简单选择排序是逐趟地对最小、次小元素进行归位，且都是最终位置，因此逐步扩充全局有序区。

【例】初始记录的关键字8 15 13 8 33 2，请写出简单选择排序法的过程。

初始序列: 8 15 13 8 33 2

↓ j

↑ 标记第一个元素

向后遍历, 找到小于标记元素的最小元素

比较的次数=5
=6-1
=n-i

第一趟排序 i=1: 2 15 13 8 33 8

↑ 标记第一个元素

向后遍历, 找到小于标记元素的最小元素

比较的次数=4
=6-2

第二趟排序 i=2: -2 8 13 15 33 8

↑ 标记第一个元素

向后遍历, 找到小于标记元素的最小元素

比较的次数=3
=6-3

第三趟排序 i=3: -2 8 8 15 33 13

↑ 标记第一个元素

向后遍历, 找到小于标记元素的最小元素

第四趟排序 i=4: -2 8 8 13 33 15

↑ 标记第一个元素

向后遍历, 找到小于标记元素的最小元素

第五趟排序 i=5: -2 8 8 13 15 33

不稳定

简单选择排序的算法 (P302) :

```
int SelectSort(RedType R[], int n){  
    //对数组R作简单选择排序  
    for(i=1;i<n;i++){ //i从1到n-1, 一共进行n-1趟排序  
        minloc=i; //假设i是最小值的下标, 让minloc记录最小元素位置  
        for (j=i+1;j<=n;j++) //在数组R[i+1...n]中找最小的元素  
            if (R[j]<R[minloc]) minloc=j; //更新最小元素位置  
        if(minloc!=i) swap(R[i],R[minloc]); //与第i个位置交换  
    } //如果该趟, i所指的正好是最小值, 则无需交换, 否则需交换  
}
```



简单选择

n 个记录中找最小记录需要 $n-1$ 次比较

简单选择排序(P302)

算法分析

- n 个记录需要 $n-1$ 趟排序。
- 第 i ($i=1\sim n-1$) 趟挑选最小记录需要比较 $n-i$ 次。

对 n 个记录进行简单选择排序，所需进行的关键字的比较次数总计为：

$$\sum_{i=1}^{n-1} (n-i) = n-1 + n-2 + \dots + 1 = \frac{n(n-1)}{2}$$

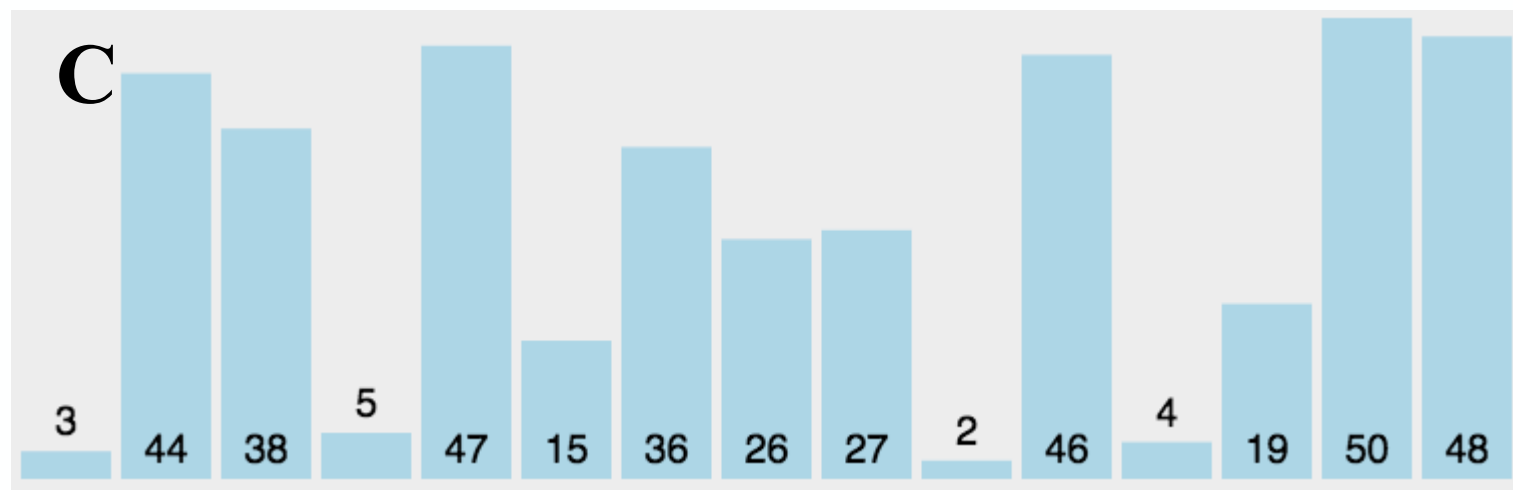
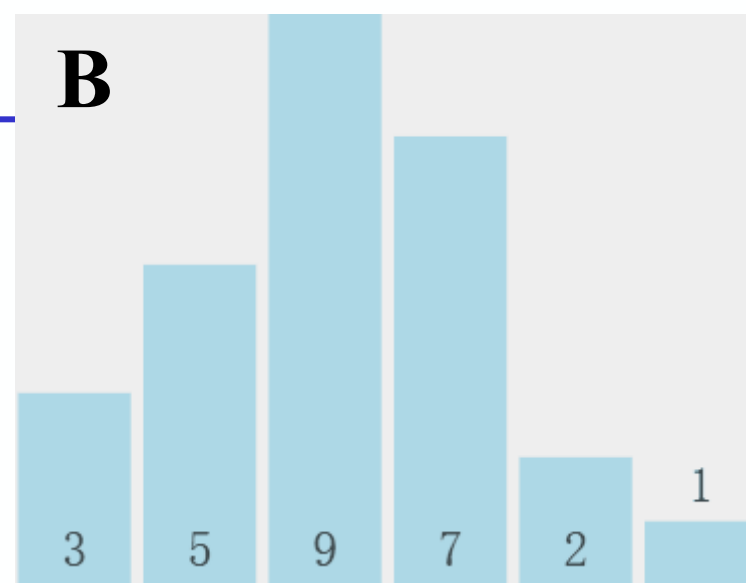
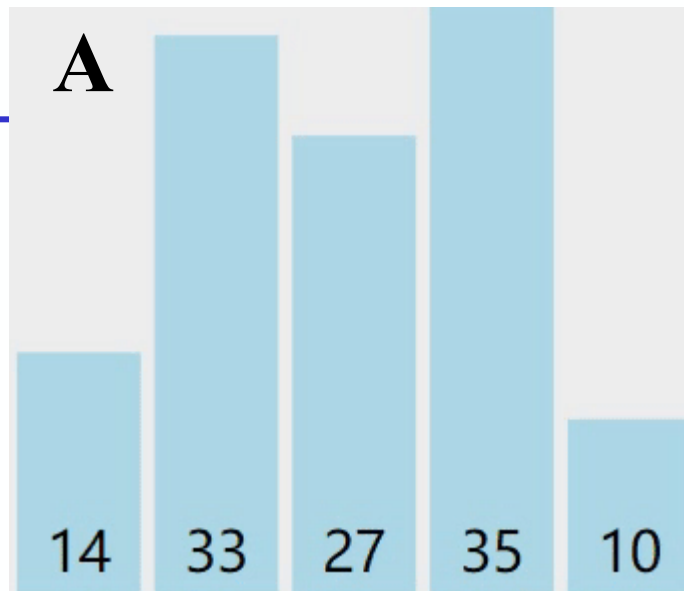
比较的时间复杂度： $O(n^2)$

每次交换都需要
移动元素3次

```
//交换  
void swap(int &a, int &b){  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

- 移动记录的次数，正序为最小值 0 ，反序为最大值 $3(n-1)$ 。

简单选择排序的最好、最坏和平均时间复杂度为 $O(n^2)$



猜测一下以上三种排序算法分别对应哪张动图？

选择排序 **B** 插入排序 **C** 冒泡排序 **A**

8.4.2 堆排序(P242)

堆的定义

若包含 n 个元素的序列 $\{a_1, a_2, \dots, a_n\}$, 满足如下性质（堆性质）：

① $a_i \leq a_{2i}$ 且 $a_i \leq a_{2i+1}$

或 ② $a_i \geq a_{2i}$ 且 $a_i \geq a_{2i+1}$ ($1 \leq i \leq \lfloor n/2 \rfloor$)

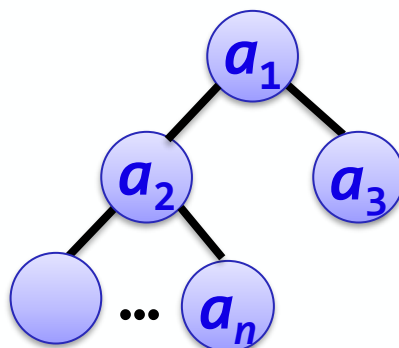
满足第①种情况的堆称为**小根堆**，满足第②种情况的堆称为**大根堆**。

特点：在排序过程中，将其看成是一棵**完全二叉树**，利用完全二叉树中双亲结点和孩子结点之间的内在关系。

完全二叉树适合**顺序存储**
(从下标1)

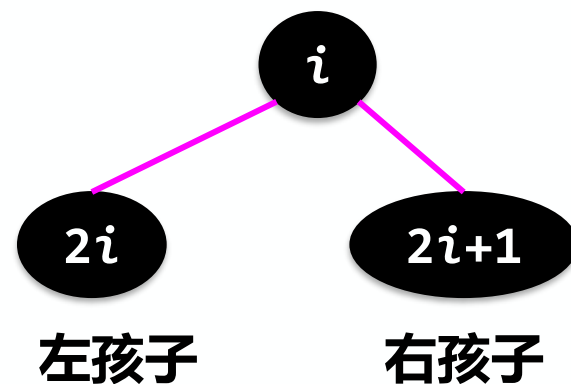
左孩子编号： $2i$

右孩子编号： $2i+1$



完全二叉树

层序编号方式：



左孩子

右孩子

任一父结点均小于等于其左右孩子是**小根堆**；
任一父结点均大于等于其左右孩子是**大根堆**。

8.4.2 堆排序(P242)

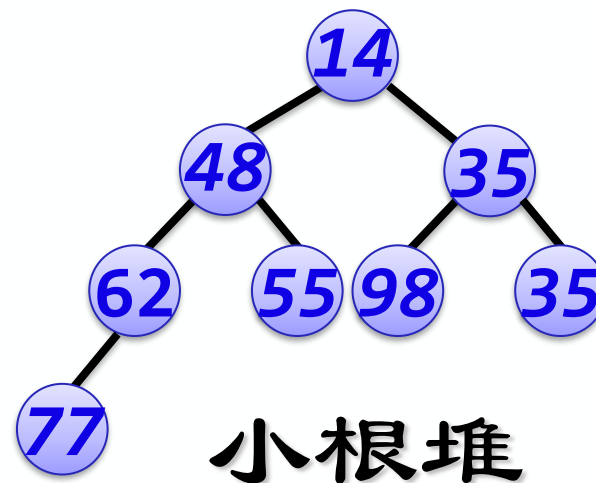
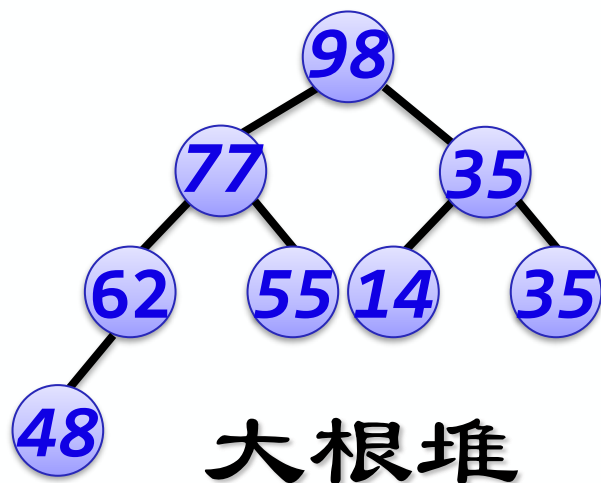
{98 77 35 62 55 14 35 48 }

{14 48 35 62 55 98 35 77 }

是堆吗?



方法：层序编号，将序列看成是一棵完全二叉树。

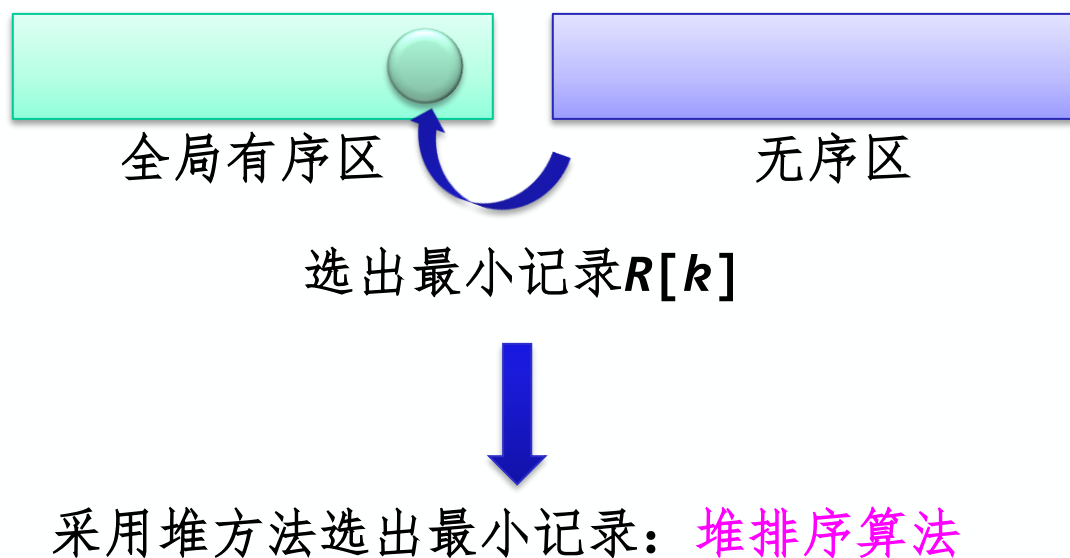


■ 大根堆：根结点就是最大值，最小值一定是某个叶结点！

■ 小根堆：根结点就是最小值，最大值一定是某个叶结点！

8.4.2 堆排序(P242)

基本思想：若在输出堆顶的最小值（最大值）后，使得剩余 $n-1$ 个元素的序列重又建成一个堆，则得到 n 个元素的次小值（次大值）...如此反复，便能得到一个有序序列，这个过程称为堆排序。



8.4.2 堆排序

1、先将初始关键字的无序序列，**建成**一个堆；

2、输出堆顶的最大（小）元素；

3、**调整**余下 $n-1$ 个元素为一个新堆，得到 n 个元素的次小值

4、重复2和3两步，直至 n 个元素输出，得到一个有序序列

拿小本记下来

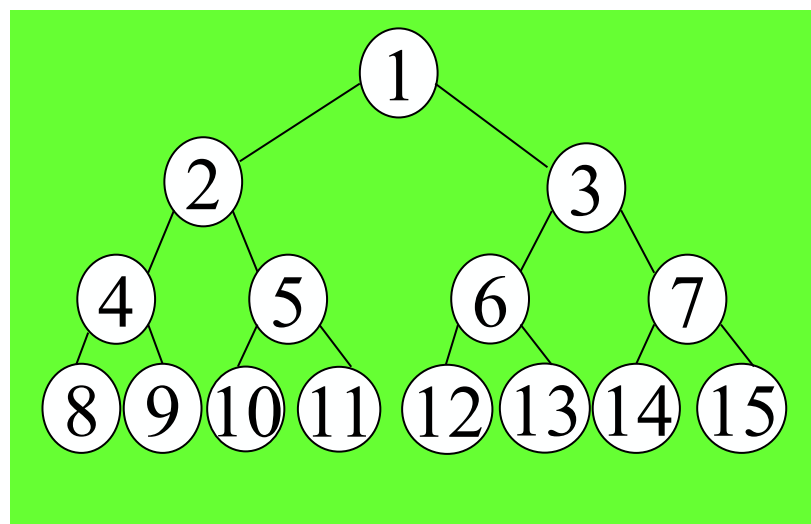


① 单结点的二叉树是堆。

② 在完全二叉树中**所有以叶结点（序号 $i > n/2$ ）为根的子树是堆**

直接从最后一个非叶结点开始调整：

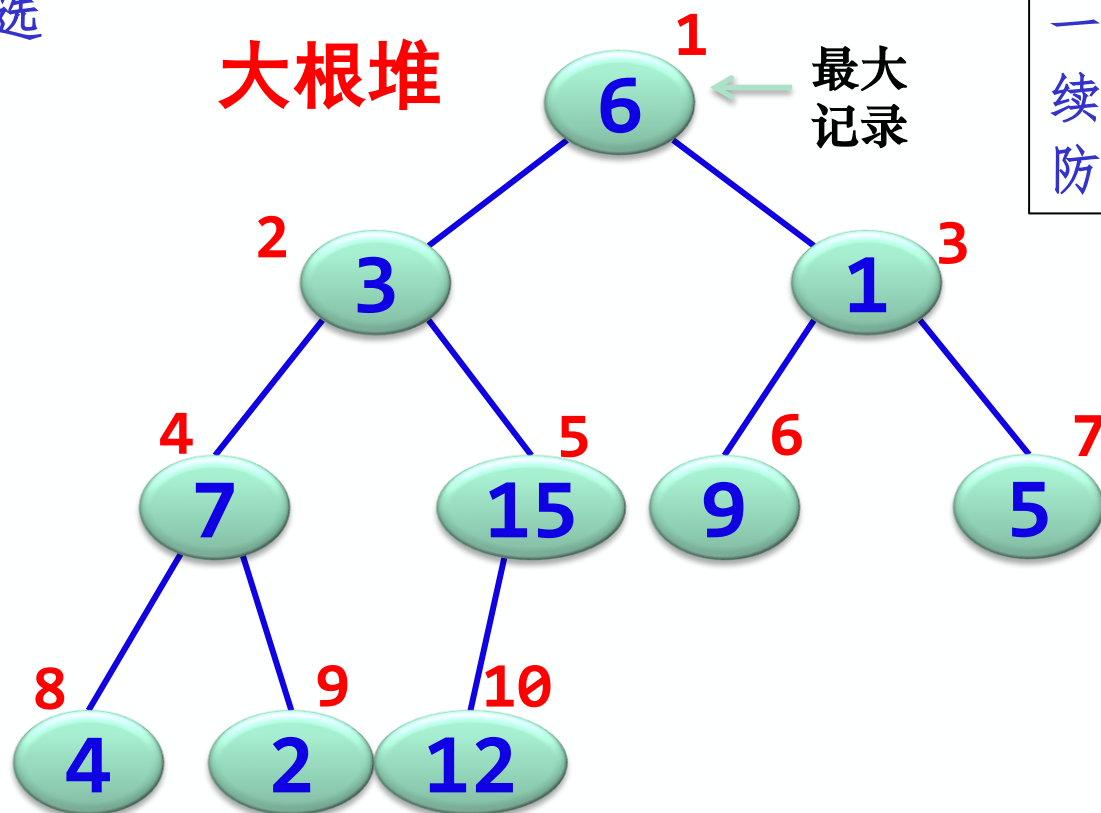
只需依次将以序号为 $\lfloor n/2 \rfloor$, $\lfloor n/2 - 1 \rfloor$,, 1的结点为根的子树均调整为堆即可。



8.4.2 堆排序

6	3	1	7	15	9	5	4	2	12
---	---	---	---	----	---	---	---	---	----

从编号为 $\lfloor n/2 \rfloor = 5$ 的结点开始，
逐一筛选



向下调整可能不止调整一次，换下去后还要继续和左右孩子比较大小防止破坏子树堆的结构

堆排序的算法 (P245) :

```
void HeapSort(RecType R[], int n)
{
    int i;
    for (i=n/2;i>=1;i--) //从最后一个非叶结点到根结点循环建立初始堆
        sift(R, i, n);

    for (i=n; i>=2; i--) //进行n-1次循环，完成堆排序
    {
        swap(R[1],R[i]); //根和最后一个元素交换,  $R[1] \Leftrightarrow R[i]$ 
        sift(R, 1, i-1); //筛选R[1]结点, 得到i-1个结点的堆
    }
}
```

堆排序

算法分析

❶ 对高度为 h 的堆，一次“筛选”所需进行的关键字比较的次数至多为 $2(h-1)$ 。

❷ 对 n 个关键字，建成高度为 h ($=\lfloor \log_2 n \rfloor + 1$) 的堆，所需进行的关键字比较的次数不超过 $4n$ 。

❸ 调整“堆顶” $n-1$ 次，总共进行的关键字比较的次数不超过：

$$2 (\lfloor \log_2(n-1) \rfloor + \lfloor \log_2(n-2) \rfloor + \dots + \log_2 2) < 2n(\lfloor \log_2 n \rfloor)$$



因此，堆排序的时间复杂度为 $O(n \log n)$ 。

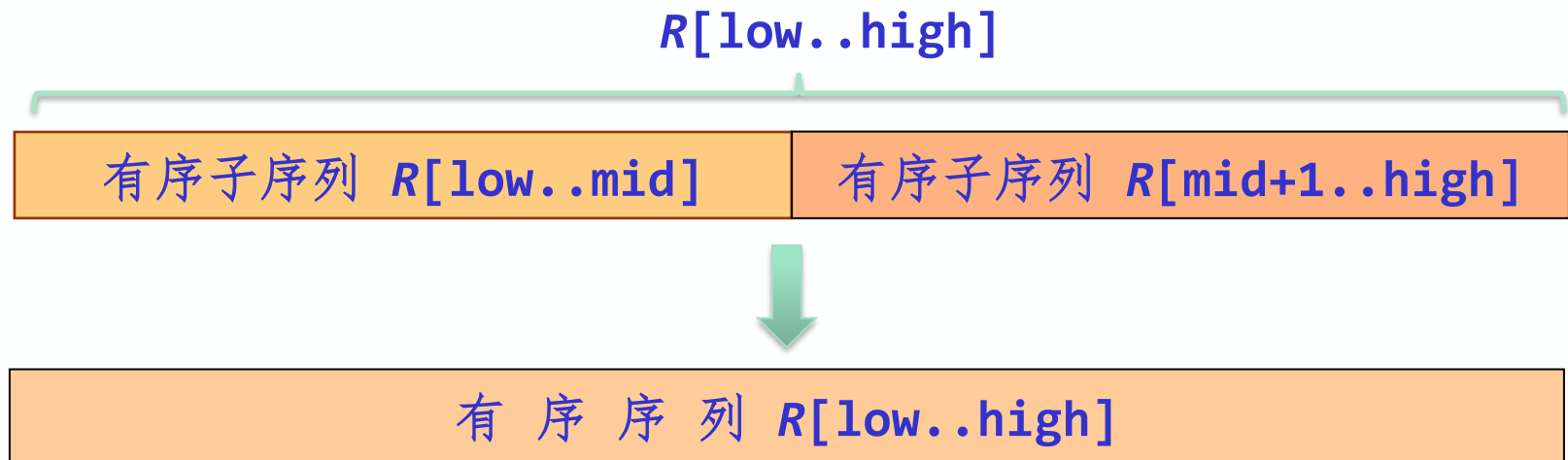
空间复杂度为 $O(1)$ ，**不稳定**。

8.5 归并排序(P247)



归并：多次将两个或两个以上的有序表组合成一个新有序表的排序方法。

2-路归并排序：多次将相邻的两个有序表合并成一个新有序表的过程，是最简单的归并排序。



【例】 设待排序表有7个元素，其关键字分别为（49，38，65，97，76，13，27）。说明采用二路归并排序方法进行排序的过程。

初始关键字： [49] [38] [65] [97] [76] [13] [27]


一趟归并后： [38 49] [65 97] [13 76] [27]

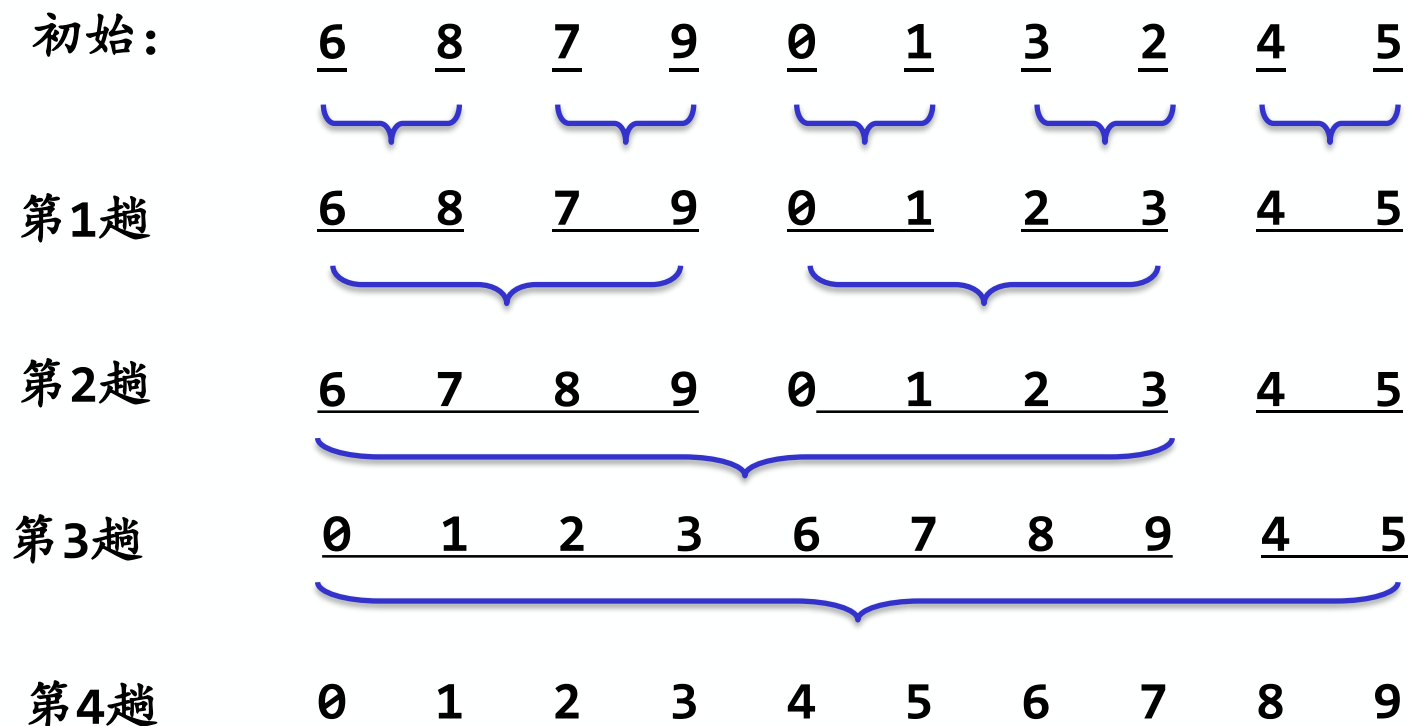

二趟归并后： [38 49 65 97] [13 27 76]


三趟归并后： [13 27 38 49 65 76 97]

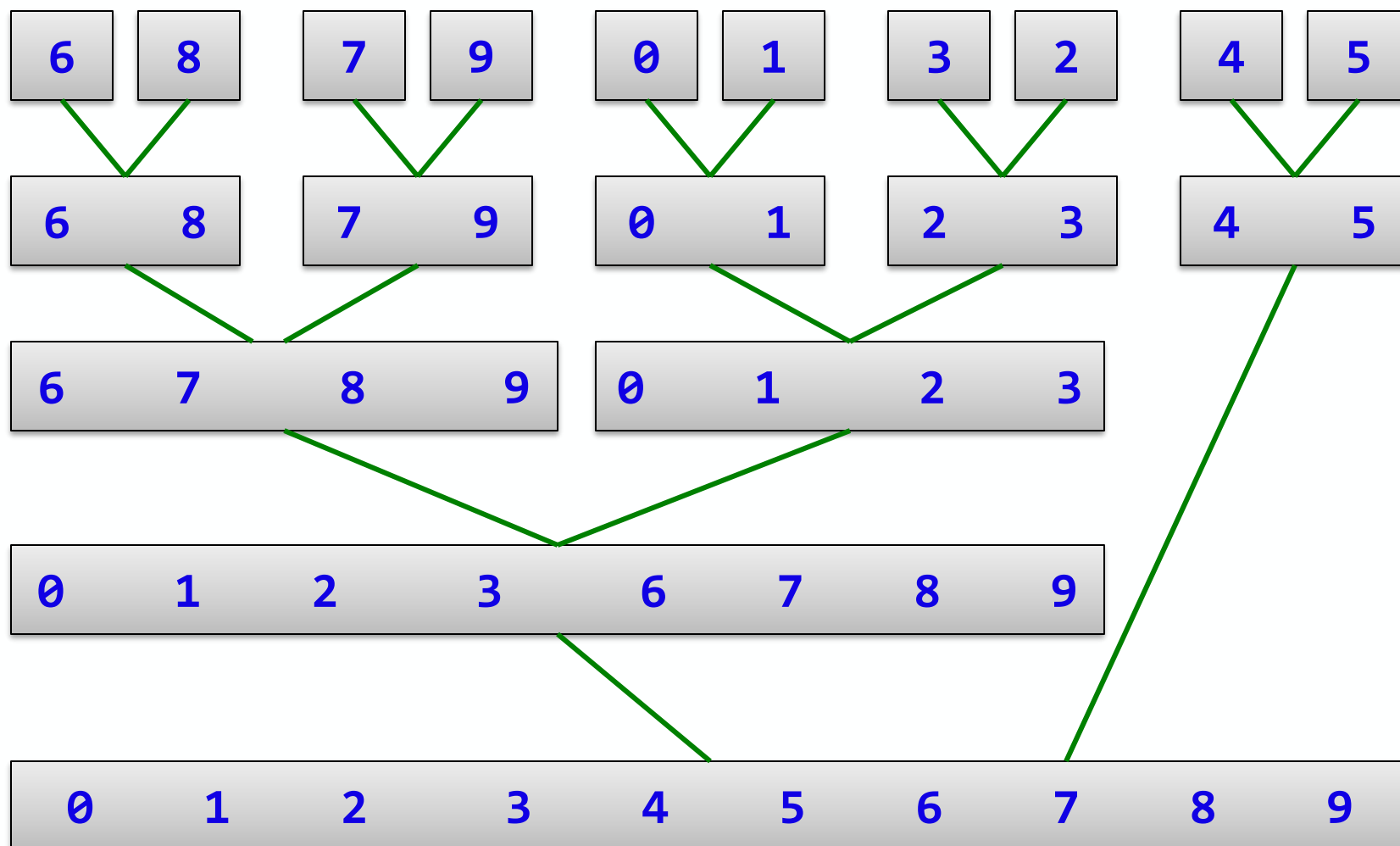
7个元素三趟归并，可以实现

趟数 $\lceil \log_2 7 \rceil = 3$

【例】 设待排序表有10个元素，其关键字分别为（6，8，7，9，0，1，3，2，4，5）。说明采用二路归并排序方法排序的过程。



需要 $\log_2 10$ 取上界即4趟



一颗归并树

8.5 归并排序(P248)

算法分析

❶ 时间复杂度分析

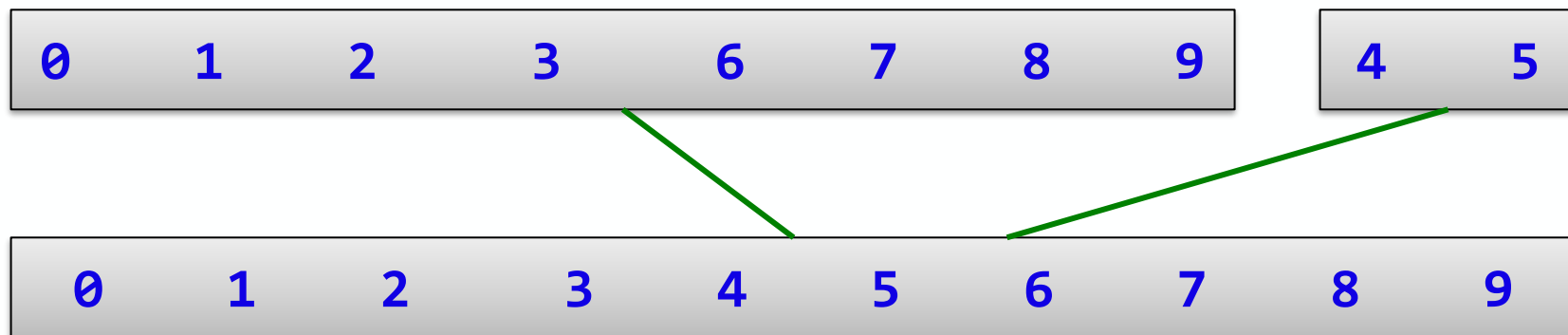
- 每一趟归并都涉及到全部的 n 个元素，时间复杂度为 $O(n)$
- 总共需进行「 $\log_2 n$ 」趟
- 二路归并排序的时间复杂度为 $O(n \log_2 n)$ 。

8.5 归并排序(P248)

算法分析

② 空间复杂度分析

每一次二路归并后临时空间都会释放。而最后的一次二路归并需要全部记录参加归并：



占用临时空间为全部记录个数 n ，所以空间复杂度为 $O(n)$ 。

■ 稳定性

归并排序是一个**稳定**的排序方法。

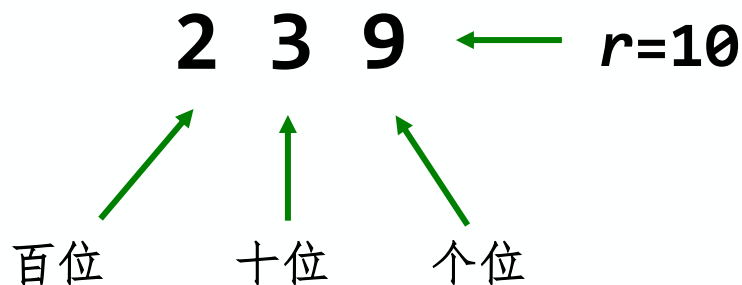


8.6 基数排序(P249)

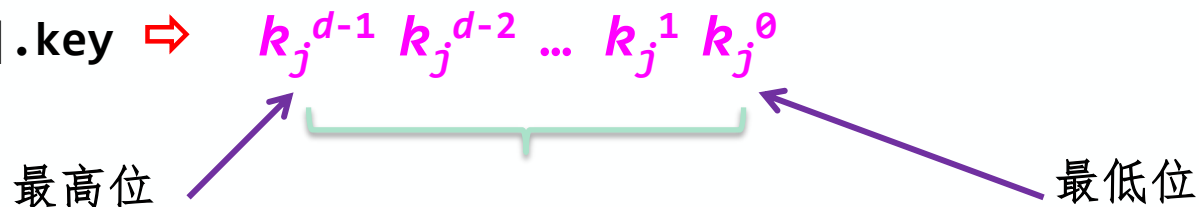
基数排序(Radix Sorting)基本思想：分配+收集

基数 r ：对于二进制数 r 为2，对于十进制数 r 为10。

示例：



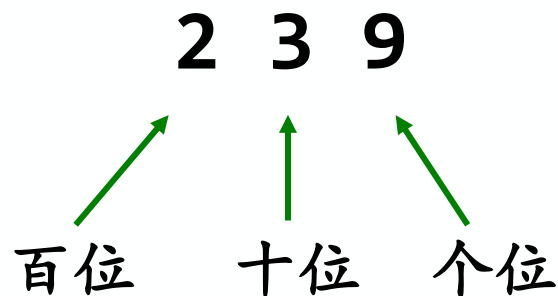
记录 $R[j]$ 的关键字 $R[j].key \Rightarrow$



该关键字由 d 位数字或字符组成
每一位的值都在 $0 \sim r-1$ 范围内

8.6 基数排序的分类

- 基数排序是一位一位地排序
- 基数排序有两种：**最低位优先（LSD）**和**最高位优先（MSD）**。



- 最低位优先：从个位 $\Rightarrow$ 百位
- 最高位优先：从百位 $\Rightarrow$ 个位

选择哪种基数排序，需要根据数据的特点来定。
例如，对整数序列递增排序，选择**最低位优先**，越重要的位越在后面排序。

最低位优先排序举例：

(369, 367, 167, 239, 237, 138, 230, 139) $\Rightarrow$ 基数排序

$p \rightarrow 369 \rightarrow 367 \rightarrow 167 \rightarrow 239 \rightarrow 237 \rightarrow 138 \rightarrow 230 \rightarrow 139$

10个元素建立10个队列， f 为队头， r 为队尾

① 进行第1次分配：按个位

$f[0] \rightarrow 230 \leftarrow r[0]$

$f[7] \rightarrow 367 \rightarrow 167 \rightarrow 237 \leftarrow r[7]$

$f[8] \rightarrow 138 \leftarrow r[8]$

$f[9] \rightarrow 369 \rightarrow 239 \rightarrow 139 \leftarrow r[9]$

进行第1次收集

- 分配时是按一个一个元素进行的
- 收集时是按一个一个队列进行的

$p \rightarrow 230 \rightarrow 367 \rightarrow 167 \rightarrow 237 \rightarrow 138 \rightarrow 369 \rightarrow 239 \rightarrow 139$

第1趟排序完毕

$p \rightarrow 230 \rightarrow 367 \rightarrow 167 \rightarrow 237 \rightarrow 138 \rightarrow 369 \rightarrow 239 \rightarrow 139$

② 进行第2次分配：按十位

$f[3] \rightarrow 230 \rightarrow 237 \rightarrow 138 \rightarrow 239 \rightarrow 139 \leftarrow r[3]$

$f[6] \rightarrow 367 \rightarrow 167 \rightarrow 369 \leftarrow r[6]$

进行第2次收集

$p \rightarrow 230 \rightarrow 237 \rightarrow 138 \rightarrow 239 \rightarrow 139 \rightarrow 367 \rightarrow 167 \rightarrow 369$

第2趟排序完毕

$p \rightarrow 230 \rightarrow 237 \rightarrow 138 \rightarrow 239 \rightarrow 139 \rightarrow 367 \rightarrow 167 \rightarrow 369$

③ 进行第3次分配：按百位

$f[1] \rightarrow 138 \rightarrow 139 \rightarrow 167 \leftarrow r[1]$

$f[2] \rightarrow 230 \rightarrow 237 \rightarrow 239 \leftarrow r[2]$

$f[3] \rightarrow 367 \rightarrow 369 \leftarrow r[3]$

进行第3次收集

$p \rightarrow 138 \rightarrow 139 \rightarrow 167 \rightarrow 230 \rightarrow 237 \rightarrow 239 \rightarrow 367 \rightarrow 369$

第3趟排序完毕

结论

基数排序是通过“分配+收集”实现排序，不需要关键字的比较

已知的信息：基数，每个位的取值，这些值的大小！



- 适合字符串排序
- 适合多关键字排序：如有数学、语文成绩，排序方式：按数学成绩递减排序，数学成绩相同按语文成绩递减排序。

基数排序算法分析

基数排序的时间复杂度为 $O(d(n+r))$

d : 关键字的位数, 决定了“分配-收集”的趟数

n : 关键字的个数

r : r 是基数, 关键字取值范围为 r 个值

其中: 分配为 $O(n)$

收集为 $O(r)$

基数排序的空间复杂度为 $O(n+r)$

基数排序稳定

主要内部排序方法的性能

排序方法	时间复杂度			空间复杂度	稳定性
	平均情况	最坏情况	最好情况		
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
折半插入排序	$O(n^2)$	$O(n^2)$	$O(n\log_2 n)$	$O(1)$	稳定
希尔排序	$O(n^{1.3})$			$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(n+r)$	$O(n+r)$	稳定

练习一下

1. 下列排序方法中，不属于内部排序的是（**C**）。
A. 插入排序 B. 选择排序 C. 拓扑排序 D. 归并排序
2. 一组记录关键字为{50, 80, 56, 29, 35, 84}，利用快速排序的方法进行第一趟排序后的结果为（**B**）。
A. 29, 35, 56, 80, 50, 84 B. 35, 29, 50, 56, 80, 84
C. 35, 29, 56, 50, 80, 84 D. 29, 35, 50, 56, 80, 84
3. 堆的形状是一棵（**C**）。
A. 二叉排序树 B. 满二叉树 C. 完全二叉树 D. 平衡二叉树

4. 快速排序在（**D**）情况下，最不利于发挥其特点。
- A. 待排序的数据量太大 B. 待排序的数据中含多个相同的值
C. 待排序的数据个数为奇数 D. 待排序的数据基本有序
5. 一组记录关键字为{25, 48, 16, 35, 79, 82, 23, 40, 36, 72}，按归并排序法进行二趟归并后的结果为（**A**）。
- A. 16, 25, 35, 48, 23, 40, 79, 82, 36, 72
B. 16, 25, 35, 48, 79, 82, 23, 36, 40, 72
C. 16, 25, 48, 35, 79, 82, 23, 36, 40, 72
D. 16, 25, 35, 48, 79, 23, 36, 40, 72, 82

6. 已知一关键字序列为{17, 18, 60, 40, 7, 32, 73, 65, 85}, 请给出采用冒泡排序法进行排序的过程。

7. 已知一关键字序列为{10, 18, 4, 3, 6, 12, 1, 9, 18, 8}, 请给出采用希尔排序法进行排序的过程。